

VASTRANGAM

Vastrangam — the rules and the logic

Everything this business runs on: every rule, every calculation, and what the system refuses.

14 parts · 54 sections · 285 rules · compiled 2026-08-28

What this document is

This describes a design. Nothing in it exists yet, and nothing in it claims to.

It is the reference, organised by subject rather than in order, because that is how somebody looks something up nine months in. *Vastrangam — the build guide* is the other half: the same business in the order you set it up.

It carries the companies, the channels, the products and what each set contains, how work is counted and paid, how attendance and pay are computed, what the system refuses to do, and all 285 rules — **each with what the system will never do instead**, which is the half you rely on when nobody is watching. Nothing is left out on the grounds that it is detail; the detail is where the money is.

Every technical word is explained the first time it appears, in plain language, with an everyday comparison. No prior knowledge is needed anywhere.

The promise this whole design keeps

You can change anything, at any time, and it takes effect at once. And the past does not move.

Every change carries the date it starts from. So a supervisor can leave on Tuesday without notice, a replacement start Wednesday morning, both recorded the same day — and last month's payroll, already paid, still comes out to the same rupee. *Purana record mitta nahin; naye date se naya rule lagta hai.*

Where you do each thing

IN THE APP	On a screen, by an administrator
ON A PHONE	By anybody, from a basic phone, in their own language
WITH YOUR TEAM	A decision or an agreement, not a screen
OUTSIDE	On somebody else's website — a marketplace, a shop platform

About people

No person is named anywhere in these documents. Names, salaries and employment details live in your system, behind permissions — not in a file that gets printed, emailed and forwarded. Every rule here is described by its shape, which is what makes it a rule rather than a list.

One thing that is never asked of you

This system will never ask you for a marketplace, bank or account password. Every outside connection uses a key you create on the other service and can withdraw from the other service. If anything ever asks you for one of those passwords, it is not this platform.

Part 9 works the changing-things case through, and lists all 18 things you can change and the 6 nobody can switch off.

Part 0 · What you are on this platform

Medhava is the software. **You are one business using it** — the same way a business uses Zoho or Odoo. You sign up, you take a plan, and you run your companies inside it.

That decides everything in this document. **You install nothing.** No server, no software on a laptop, no technical person needed. Everything here happens in a browser or on a phone.

The businesses on either side of you look nothing like yours — a steel plant, a school, somebody selling courses. Same software underneath. What makes it yours is the settings: your words, your steps, your companies, your channels, and which parts of it you use at all.

platform — One piece of software that many separate businesses use at the same time, each seeing only its own information. *Ek badi building jisme bahut saare offices hain. Building ek hai, par har office ki chaabi alag — koi kisi aur ke office mein nahin ghus sakta.* **tenant** — One business using the platform. Its people, its data and its settings are its own. *Us building mein ek office. Aapka office, aapka saamaan, aapka taala.* **module** — One area of work in the system — sales, purchase, staff, accounts. Each is a set of screens that belong together. *Dukaan ke alag-alag counters. Ek counter bikri ka, ek kharidi ka, ek hisaab-kitaab ka.* **role** — What a person is allowed to see and do — a manager sees more than a counter staff member. *Chaabi*

ka guccha. Manager ke paas zyada chaabiyaan, staff ke paas kam. permission — One specific thing a role is allowed to do, like approving a discount or viewing salaries. Guchhe ki ek chaabi. Ek chaabi ek darwaza.

0.1 · Understand the one thing that makes this different **WITH YOUR TEAM**

Most business software gives you a fixed system and a support ticket. Here, the things that make your business yours are **settings you control**, not code somebody has to change. That means you are never waiting on a developer to run your business — and it also means the settings are your responsibility to get right.

You change this yourself, any time	This is the same for every business
What you call everything	That every record names its company
The steps your work moves through	That money is exact to the paisa
Extra fields on any record	That every change is recorded, permanently
Which parts of the system you use	That nobody else can read your data
Your companies, channels, godowns	The rules your books rely on
Your rates, your people, your roles	That nothing is ever truly deleted

Done when: Whoever will administer this can say which column any given thing falls into.

0.2 · Know which two addresses do what **WITH YOUR TEAM**

Easy to confuse, and confusing them wastes a day. **vastrangam.com** is your shop — where customers browse and buy. It is one of the ways you sell, and orders from it arrive here. **The platform is where you run the business** — every order from every channel, the making, the stock, the people, the books.

Done when: Everyone understands the shop is one channel feeding the system, not the system itself.

Part 1 · Your companies

A company is a record you create. You have three today; the plan allows twenty, and the software itself has no limit. A fourth opens the day you open it — no waiting, no upgrade, no call.

Four things about a company are separate fields on purpose, and collapsing any two of them is the single most common modelling mistake in this whole system.

Legal name	Trades as	Brand code	Invoice prefix
Vastrangam	Vastrangam	VS	VS
Ethnic Fashion	Go4Fashion	EF	EF
Adini	Adini Couture	AC	AC

1.1 · Create each company with its four identities kept apart **IN THE APP**

Look at the middle line of the list above. The company has one legal name, trades under a different name, marks its stock with a third code, and numbers its invoices with a fourth. If any two of those were one field, its invoices would carry a name that is not its registered name — which is a compliance problem, not a cosmetic one.

Field	What it is	Where it shows
Legal name	The registered entity	Invoices, returns, contracts
Trading name	The name customers know	The shop, the packaging, marketing
Brand code	The short code on stock	SKUs, labels, stock reports
Invoice prefix	The letters before every document number	Invoice and voucher numbers

Changing it: Add a company any time. Rename one any time — documents already issued keep the name they were issued under.

Done when: Every company exists with all four set separately, and a test invoice from each carries the right name and the right number.

1.2 · Handle the company that has no registration of its own **IN THE APP**

One of your companies does job work and has no registration. It still belongs in the group figures — the work is real and the cost is real — but it must never be pulled into a return it does not belong in. Those are two different questions and the system must answer them separately.

Changing it: Whether a company files its own returns is a setting on the company, changeable from a date — so the day it does get registered, you set it and nothing before that date moves.

Done when: The group total includes it, and no return anywhere includes it.

1.3 · Check that trade between your own companies is removed from the group figure **IN THE APP**

Selling stock from one of your companies to another is not group revenue. Counting it means the group looks bigger than it is, and you are the person that number misleads.

Done when: The group total equals the sum of the companies minus trade between them, checked against one such transfer you can point at.

Part 2 · Your channels — every way you sell

A channel is where a sale came from. You record one per company, so two of your companies can each sell on the same marketplace and they stay separate, with figures that never merge.

Stock stays one number per item. Never split per channel. That single decision is what stops the same piece being sold twice on two different marketplaces.

Kind	What it is
d2c	Your own shop — vastrangam.com is this one
marketplace	A marketplace account. One for each marketplace, for each company
b2b	Wholesale, usually on credit terms
export	Overseas, with its own documents
pos	A counter, drawing on the same stock as the shop
reseller	Somebody selling on your behalf

A channel is where a **sale** came from. Where an **enquiry** came from is a different question with a shorter answer, and you have already fixed the list:

Where a lead came from
IndiaMART
Website
WhatsApp
Walk-in
Forum

Closed today, not closed forever. The list is closed today, not closed forever. A tenant adds an entry through `core/tenant.js` like any other change: effective-dated, append-only, and the closed months keep the list that applied then. What 'locked' rules out is a value appearing in a record that is not on the list — a lead whose source is a free-text guess, or a service sent to a vendor nobody appointed.

2.1 · Add every route to market you actually use **IN THE APP**

Changing it: A new marketplace is added in the app and selling the same day. Never a release, never a wait.

Done when: Every way you currently sell is recorded against the company that owns it.

2.2 · Connect each selling account with a key, never a password **OUTSIDE**

Every connection uses a key you create in that marketplace or shop, and can withdraw at any time without changing anything else. **Nothing in this system will ever ask you for a marketplace, bank or account password** — a password hands over an account you cannot take back or limit. If anything ever asks, it is not us.

Done when: Every channel is connected with its own key, and you know where to withdraw each one.

2.3 · Decide what happens when a channel goes quiet **IN THE APP**

Marketplaces change their systems without telling anyone. When orders stop arriving from one, that must be visible rather than silent — a channel that quietly stops feeding is a week of missing sales nobody noticed.

Done when: A channel that has sent nothing for longer than it usually does raises a flag somebody sees.

Part 3 · Your products, and what each set actually contains

This is the part most systems get wrong, and getting it wrong changes what you pay people.

You sell **sets**. A set is several pieces that go together, and different sets contain different things. The critical fact: **you cannot tell what a set contains from its name**. An Anarkali Plazo Set contains a dupatta. A Kurti Plazo Set does not. Neither name says so. Read the composition from the name and you get one of them wrong whichever way you read it.

Set type	What it contains	Designs checked	If a piece is missing
Anarkali Plazo Set	Top + Bottom + Dupatta	41	your decision — both counts shown
Kurti Plazo Set	Top + Bottom	16	your decision — both counts shown
Kurti Palazzo Set	Top + Bottom	25	your decision — both counts shown
Lehenga Choli Set	Top + Bottom + Dupatta	34	Top: your decision · Bottom: your decision · Dupatta: optional
Co-Ords Set	Top + Bottom	2	your decision — both counts shown
Top Set	Top	24	your decision — both counts shown
Bottom Wear Set	Bottom	8	your decision — both counts shown
Dupatta Set	Dupatta	2	your decision — both counts shown
Kurta Set	Top	1	your decision — both counts shown
Readymade Blouse Set	Top	1	your decision — both counts shown

These were not read off the names. Each one was checked against real production records until only one composition reproduced every design. Two of them prove why that mattered:

- **Anarkali Plazo Set** — only Top+Bottom+Dupatta reproduces all 41. Taking Top+Bottom instead reports 1,400 sets for BinaRust where the file records 1,225.
- **Kurti Plazo Set** — only Top+Bottom reproduces all 16. Including the dupatta reports 194 sets for GreenKurtiPlazzo where the file records 854.

Your files are not tidy, and they should not have to be. Two rules do the heavy lifting.

A heading is only a heading if there is a date under it

Structural, not cosmetic — so it works whether or not somebody has tidied the sheet. A stray heading sitting on top of another heading is recognised as stray, instead of quietly eating a month of data underneath it.

Columns are found by name, never by position

Columns move when somebody joins or leaves. People do not. Reading by position means the day a column is inserted, every figure after it belongs to the wrong person — and nothing about that looks wrong on screen. Reading by name means you can add a column, remove one, or reorder the whole sheet and nothing breaks.

The same rule governs the master workbook: it is read **by column name**, so your business can add a column of its own whenever it needs one.

Where personal and banking details stop

Identity numbers, bank name, account number, IFSC, UPI, phone and address are read into a **separate object that nothing else writes to disk**. Personal and banking data **never leaves** the module that reads it, and none of it is attached to the master record the rest of the system passes around.

So a report, an export, a backup of the working data or a file sent to somebody cannot carry them, because they were never in it. That is a stronger guarantee than a permission, which somebody can be granted.

3.1 · Record what each set type contains, as a list of slots **IN THE APP**

Because the composition decides how many complete sets exist, and how many complete sets decides what gets paid. This is not a description — it is an input to a payment.

Changing it: Add a set type, change what one contains, or retire one — any time, from a date. Sets already counted keep the composition that applied when they were counted.

Done when: Every set type you make has its slots recorded, and none of them was guessed from its name.

3.2 · Understand the three separate dupatta columns **WITH YOUR TEAM**

Your production report has **three different dupatta columns** — one for Anarkali Plazo, one for Kurti Palazzo, one for Lehenga Choli — plus a standalone Dupatta Set column. A system with a single dupatta slot cannot tell them apart, and would credit a Kurti Plazo design with a dupatta it never had. The columns are separate because the garments are separate.

Done when: Whoever fills the production report knows which dupatta column belongs to which set type.

3.3 · Know where the set type for a design comes from, and what happens when it is missing **WITH YOUR TEAM**

The set type for a design comes from your rates master. When a design has no entry there, the system works it out from which columns have numbers in them — checking in a fixed order, most specific first — and **flags the result as worked out rather than known**. A flag is not a failure; it is the system refusing to pretend it was told something it inferred.

The set type for a design	How it is decided
Where it normally comes from	Your rates master — the design, its set, its attribute and its rate
When that has no entry	Worked out from which columns have numbers, checked most specific first
The order checked	Lehenga · Anarkali · Kurti Palazzo · Kurti Plazo · Co-Ords · single column
What then happens	The result is flagged as worked out , never presented as known
Columns in the report	22, arranged in groups by set category

The layout matters when somebody fills it in: Row 1 title (skip) · Row 2 set-category group labels (skip) · Row 3 garment-type labels · data from Row 4. Col A = Karigar, Col B = Design, Col C onward = the 23 columns.

Changing it: Add the design to the rates master and the flag disappears from that point on. Nothing already counted changes.

Done when: Every design either has a set type on record or is flagged as inferred, and somebody reviews the flagged list.

3.4 · Carry the column-count discrepancy openly rather than resolving it by guess **WITH YOUR TEAM**

Your own written specification says 23 garment columns in two places, and then lists 22 — indices 2 to 23, columns C to X — with none unused. The system holds the 22 that are actually named. If a 23rd exists in the real file it has never been named anywhere, and quietly inventing one would silently mis-file whatever it holds.

Done when: Somebody who knows the original file confirms whether 22 is right. Until then it stays flagged, not smoothed over.

Part 4 · The making side — counting and paying for work

The heart of the business, and the part with the most rules. Read this part slowly; every line of it turns into somebody's payment.

A name written in capitals, in mixed case, with a trailing space, or with one letter transposed is still the same person. So **the system never compares written names**. It compares ids: names in, **one identity** out. There is exactly one place where a written name becomes an id.

What is compared	The id. Never the spelling
Where a name becomes an id	One place, and only one
Where the spellings live	An alias table — which is data you edit, not code
An exact alias	Resolves silently. It is already your answer, given earlier
A near match	Proposed, never applied. A merge is a decision
Once you decide	Stored. You are asked once, not every month

Merging two people who are actually different silently combines two balances, and separating them afterwards means unpicking every payment either of them ever received. That is why the system will not do it on your behalf, however confident the match looks.

Which slot a component fills is worked out from structure, not from its name

A rate card lists components. Which piece of a set each one is — the top, the bottom, the dupatta — is decided from the structure of the card itself, not by reading the label and hoping. A label is what somebody typed; structure is what the data actually shows.

What was actually paid, weighted by quantity

A design may have been paid at more than one rate across a period. The rate reported is **weighted by quantity**, not a plain average of the rates — twenty pieces at one rate and two at another is not the midpoint of the two rates, and treating it as such misstates the cost of every one of those twenty-two pieces.

When two periods disagree about a rate

The later one wins, and both are kept. The recent decision is the operative one, and the earlier one stays visible so a report for the earlier period still resolves what applied then — and so the disagreement itself is on the record rather than silently resolved.

The adjustment that makes a design tie out

Where the computed figure and the raw recorded figure differ, the difference is carried as a **named adjustment** rather than being absorbed. A design that ties out because somebody quietly nudged it is a design nobody can check.

Everything lands on the unit, whatever it was called that period

Earnings, payments and the outstanding balance all roll up to the paying unit — through whatever label the source used in that period. One unit, one continuous balance, however many names it has been written under.

Nothing is read off a total

Every figure is **recomputed from the transaction rows**. The totals your own source files carry are used only to check the answer — never as the answer. And where the two disagree, the difference is reported rather than reconciled away.

4.1 · Understand that the paying unit is not the same as the person WITH YOUR TEAM

Some units on your payroll are one person. Some are a team working under one name. The unit is what earns, gets paid and carries an outstanding balance — and a unit that worked alone one year and as a team the next is **the same unit with two labels**, each with the date it applied from. Treating them as two units would split one balance in half.

Changing it: A unit can change its label and its members from a date. Both the old and the new are kept, so a report for either period shows what it was called then.

Done when: Every paying unit exists once, with its label history, and its outstanding balance is continuous across a change of composition.

4.2 · Count a complete set as the smallest of its slots — never the smallest of whatever was made WITH YOUR TEAM

The most important calculation in the business. If a set needs a top, a bottom and a dupatta, then 100 tops, 90 bottoms and 40 dupattas is **40 complete sets** — the dupatta is the bottleneck. Counting the smallest of whatever happened to be produced, instead of the smallest of what the set actually requires, gets a different answer for any design where a slot was never populated at all.

Made	Top	Bottom	Dupatta	Complete sets
Anarkali Plazo Set — needs all three	100	90	40	40
Kurti Plazo Set — needs two	100	90	— <i>(not part of this set)</i>	90

Done when: The set count for every design uses that design's recorded composition, and a check proves it rather than assuming it.

4.3 · Price every row from its own rate, and refuse the row that has none IN THE APP

Every line of work carries its own rate, resolved for the date it was done. When a rate is missing, the system **stops and names the row** — it does not fall back to a similar rate, an average, or zero. A guessed rate is a wrong payment to a real person, and it is discovered weeks later by that person.

Changing it: Rates are changed from a date. Work already priced keeps the rate that applied when it was done.

Done when: A row with no rate blocks the run and names itself. Nothing is ever priced at zero for want of a rate.

4.4 · Never merge two similar names automatically WITH YOUR TEAM

Two spellings of a name might be one unit or might be two. The system proposes a match and **never applies one by itself**, because merging two people who are different silently combines two balances and is very hard to untangle afterwards. Once you answer, the answer is stored — so you are asked once, not every month.

Done when: Every proposed merge was decided by a person, and the decision is recorded so it is never asked twice.

Part 5 · People, attendance and pay

Three ways of being paid, and the rules that keep them apart. **Nobody is named in this document** — the roster is yours and stays in your system, not in a file that gets emailed around.

3 ways of being paid, and a person can move between them — from a date, never backwards by accident.

Basis	How the figure is reached
Attendance	Resolved from the days and the attendance recorded for the period, against the rate in force on those dates.
Flat	A fixed amount for the period, whatever the hours. Hours are recorded and reported, and never scale the pay.
Piece-rate	Defined by your own rules for this basis.

Each person's basis is held as a small history — what it became, and the date it started applying — so asking "what was this person on in March" has an exact answer rather than requiring somebody to remember.

7 codes. Each one carries two separate numbers, and keeping them separate is the whole point of this table.

Code	Means	Counts for pay	Counts for work
P	Present	1.0	1.0
H	Half day	0.5	0.5
HL	Holiday	1.0	0.0
OD	On duty, offsite	1.0	1.0
PL	Paid leave	1.0	0.0
UL	Unpaid leave	0.0	0.0
A	Absent	0.0	0.0

Look at Holiday and Paid leave. Both count a full day for pay and **zero** for work. That is not a rounding choice — it is the truth, and it is why this system has a figure called unallocated labour instead of quietly spreading a holiday across the designs somebody made that week. Paid is not the same as productive, and a system that merges them overstates how expensive every design was.

About 26 written forms are understood — a person may write **P**, **Present**, **1** or **Full** and all four mean the same thing. What is deliberately NOT a code is a **blank cell**: a blank is a state, not a mark, and what it means is decided from the employment dates rather than guessed.

Read code is the step that turns what somebody wrote into one of the codes above. Anything it does not recognise **stops the run and names the cell** — never guessed at, never treated as absent.

Day type decides whether a date is a weekday or a Sunday for that person, because the shift length differs and so does what a mark on it is worth. It is resolved from the calendar, not from whoever is entering the data.

The code table is data. Add a code, change what one is worth, or override the whole table for your business — every calculation downstream follows, because none of them names a code.

4 ways of being paid. A person can move between them, from a date.

Basis	How the month is calculated
Flat	The full monthly salary, every month, whatever the attendance . No scaling up and none down.
Attendance	The daily rate times the days worked. Uncapped in both directions — 30 days against a 27-day threshold pays for 30, and 20 days pays for 20.
Daily-wage	A daily rate that is simply stated. There is no monthly salary to divide and no threshold to divide it by.
Piece-rate	Output times the rate. No salary, no threshold, no attendance row enters this calculation at all.

Month pay is the name of the one figure that pays a person for a month. There is exactly one, per person, per month — and everything downstream reads it and nothing else. Two ways of pricing the same month is how two departments arrive at two different totals.

The order month pay is resolved in

It stops at the first thing that fails, and says which — it never carries on with a substitute.

#	It asks	If there is no answer
1	Was this person employed in this month at all?	Stop. This is not an absence and not a gap — they were not there
2	Which basis was in force, on these dates?	Stop and say so. Never fall back to a default basis
3	What was the salary, the threshold in days, the threshold in hours?	Stop and name what is missing. Never treat a missing value as zero
4	What attendance was marked?	A code with no hours defined stops this month and says why
5	Which of the states does this month fall into?	See the table below

The four states a month can be in

State	What it means	Why it is separate
Employed	Employed, and attendance was recorded	The normal case
No data	Employed, and nothing was recorded	A tracking gap worth chasing
Not employed	Outside their employment dates	Not a failure. They were not there, so it is not a missing month
Unresolvable	Something needed was missing	Reported by name, never silently priced at zero

Merging these four is how a report ends up listing people as having failed months they never worked in.

The daily rate, the hourly rate, and which one may pay anybody

Figure	How it is reached	What it may be used for
Daily rate	The salary divided by the threshold in days	Paying an attendance-based month
Hourly rate	The daily rate divided by that person's own shift length	The work report only. It pays nobody
Blended daily	The average of that year's monthly daily rates, across employed months only	Costing a design across a year
Blended hourly	The blended daily rate over the shift length — or, for piece-rate people, their stated rate per hour	Costing hours onto designs

Employed months only, and that word carries weight. Averaging a twelve-month window across an eight-month spell understates the rate by a third — and that understated rate then understates every design that person ever touched. A month somebody was not employed has no daily rate to average. It is not a zero. It is not a month.

Shift length is per group, read from a table — not written into the formula. A business whose working day is not the same as yours changes one table and nothing else.

Piece rate wage is its own calculation: hours logged against designs times that person's flat rate per hour. No threshold, no attendance, no scaling — the rate is given outright.

FY pay is the same person across one whole financial year, month by month. Which leads directly to the rule below.

Staff pay does not add up across financial years

Refused outright, rather than allowed and quietly wrong. Piece-rate earnings **are** additive across years — the same design earns the same way whenever it was made. Staff pay is not, because the threshold, the salary and the basis are all specific to a period. A combined threshold across two years is not a bigger number or a smaller one; it is a meaningless one.

Total payroll covers every person, every month, every basis — nothing excluded quietly. A piece-rate contractor whose hours are charged to designs but whose wage is left out of the payroll makes unallocated labour look smaller than it really is.

Hours get logged against designs. Costing them means turning hours into money and putting that money onto the design that consumed it — **and showing plainly what would not go anywhere.**

Utilisation — two of them, never merged

Measure	How it is reached	What it tells you
By days	Days worked against the threshold in days	Whether the month was a full one
By hours	Productive hours against the threshold in hours	Whether those days were productive

Somebody can be at a hundred per cent on days and well under on hours — a month full of holidays and paid leave does exactly that. One number would hide it.

Unallocated labour, and why it stays visible

Every hour that could not be attached to a design stays in its own figure. Holidays, paid leave, time on nothing in particular, anybody whose rate could not be resolved. **It is never spread across the designs to make the sum come out neat.**

Spreading it would make every design look slightly more expensive than it was, and would hide the one number worth acting on: how much paid time is not reaching product.

Cost per piece

The **cost per piece table** carries one line per design and nothing else: hours logged, money those hours cost, quantity made, and the cost of one piece. And one rule about the shape of it — **there is no TOTAL row**. A totals row sitting in the same table as the detail rows is how a cost report gets added to itself and reports double.

Any row whose person has no resolvable rate is listed separately by name of design and hours, rather than being costed at zero and blended into the average.

People are banded against the average — and the whole difficulty is deciding which months belong in that average at all.

Band	What it means
Satisfactory	Measured against the average of the months that count
Average	Measured against the average of the months that count
Below Average	Measured against the average of the months that count

Which months count

Month	In the average?	Scored as
Employed, with attendance	Yes	Its actual band
Employed, nothing recorded	No	No Data — called what it is
Outside their employment dates	No	Nothing. They were not there

Neither of the last two is ever scored as below average, and that is not a kindness — it is accuracy. A month somebody did not work is not a month they worked badly, and averaging it in as a zero produces a number that is wrong about a real person, on a record that follows them.

5.1 • Set each person's pay basis, from a date **IN THE APP**

A person can move from one basis to another — and when they do, the change applies from its date and not before. A basis that applied backwards would rewrite months already paid.

Changing it: Change a basis any time, choosing the date it starts. Every earlier month recalculates to the basis that applied then.

Done when: Running last month twice, before and after a basis change, gives the same figure both times.

5.2 · Keep hours informational — never let them scale a month's pay **WITH YOUR TEAM**

Hours are recorded and reported because they are useful to know. They do not multiply anybody's pay. There is exactly one figure that pays, per basis, and everything downstream — allocation, reconciliation, outstanding — reads that one figure and nothing else. Two ways of pricing the same month is how two departments end up with two different totals.

Done when: One paying figure per person per month, and every downstream report traces to it.

5.3 · Treat "no match" as an error, never as zero **WITH YOUR TEAM**

The single most dangerous default in payroll software. When the system cannot find what applied — no salary on record, no rate for a category, no hours reference — it must stop and say so. Treating a missing value as zero pays somebody nothing and looks exactly like a correct run.

Done when: Every missing value stops the run and names what is missing and for whom. No run completes with a silent zero in it.

5.4 · Separate the three states a blank month can mean **WITH YOUR TEAM**

A month with nothing recorded can mean three completely different things, and merging them misreports people badly: the person was not employed then, or they were employed and the record is missing, or they were employed and genuinely did no work. Only the middle one is a problem to chase.

A blank month means	What it is	What to do
Outside their employment dates	Not a gap at all	Nothing — they were not there
Inside employment, nothing recorded	A tracking gap	Find out what happened
Inside employment, recorded as none	A real zero	Nothing — it is the truth

Done when: The three are reported separately, and nobody appears in a failure list for a month they were not employed.

5.5 · Record advances against the unit, and settle them at payout **IN THE APP**

An advance is money already given. It has to reduce what is due without disappearing from the record, so both the person and you can see what was taken and what remains.

Changing it: Record an advance at any moment, from a phone. It affects the next payout immediately.

Done when: Every payout shows what was earned, what was advanced, and what was paid — and the three reconcile.

5.6 · Let the shop floor report without opening a computer **ON A PHONE**

The people making the product do not sit at a desk. A short message becomes a real record: attendance with the time and place it was marked, production against the design, a request in the approvals list. Anything that requires a laptop simply does not get recorded, and ends up on paper.

Careful. Attendance marked outside the unit's area is **flagged for a manager, never refused**. A system that locks somebody out of being paid because they stood at the wrong gate has failed at its job. Every override is recorded with who made it.

Done when: A worker can mark attendance and report production from a basic phone, in their own language.

5.11 · Write down the holiday and festival rules, and keep paid separate from productive **WITH YOUR TEAM**

Every business has these and almost none of them are written down, which means they are whatever the person running payroll remembers. Written down they are settings; remembered they are a different answer each year, and the difference lands on somebody's wages.

The distinction that carries the whole subject is paid against productive. A holiday is a full day of pay and zero hours of production. Those are two different numbers and a system that treats them as one flatters every efficiency figure that reads them — the factory looks more productive on the days nobody worked.

The day is marked	It pays	It produces
Present	A full day	A full day's hours
Half day	Half a day	Half a day's hours
Holiday	A full day	Nothing — nobody was making anything
On duty, offsite	A full day	A full day — the work happened elsewhere
Paid leave	A full day	Nothing
Unpaid leave	Nothing	Nothing
Absent	Nothing	Nothing

Changing it: The holiday list, which days are paid, and who each rule applies to are settings with dates. Adding Diwali to next year's list does not re-run last year's payroll.

A festival flag matches a festival-leave request and does nothing else. It is not a pay rule, not a shift rule and not a permission — and the reason it is worth saying is that a flag attached to a person is exactly the kind of thing that quietly acquires a second job later, at which point somebody's pay depends on a field nobody thought was about pay.

Careful. A festival spike in demand is **never folded into the baseline**. A month that sold three times the usual amount because of one festival is not a new normal, and averaging it in has the system ordering for a festival all year.

Done when: Holiday and festival rules exist as dated settings rather than in somebody's memory, a paid day that produced nothing is counted as paid and not as productive, and no festival month has been averaged into a baseline.

5.13 · End a rate on the date it ended, and let the next period ask rather than assume [IN THE APP](#)

A rate left running with no end date does not stay put — it keeps applying. Somebody who was paid by the hour last year and moves to piece work this year has **two periods**, and if the hourly row is never closed, this year quietly pays last year's figure. It reconciles, it posts, and nobody sees it until somebody adds up what they were actually paid.

Naming the new basis is not the same as stating the new rate. Saying "from April it is piece rate" says what kind of pay it is and not how much, and those are two facts. The second one has to be given before anybody can be paid under it.

What is on record	What the system does
A rate with an end date	Applies up to that date and no further
The period after it, with a rate	Pays the new rate
The period after it, with no rate	Raises, and names the person and the month. It does not carry the old rate forward and it does not pay zero.
Somebody who never had a rate at all	The same refusal, recorded in its own place — a rate that ended is a different fact from one that never existed

Changing it: Close a rate from a date and add the next one whenever you know it. A month between the two raises until you do, which is the system asking rather than choosing.

Careful. *This is not a warning about carelessness. It is a warning about the shape of the mistake: the wrong answer here **looks completely normal**. Last year's rate is a real number that was correct twelve months ago, so the payslip is plausible, the ledger balances, and the only person who can tell is the one being paid.*

Done when: Every rate has an end date once it has ended, a period with no rate stated raises and names who and when, and no closed month ever changed because a later rate was added.

5.12 · Record religion where a holiday depends on it, and let it decide nothing else [IN THE APP](#)

Some observances apply to everybody and some do not, and the only honest way to work out who a religion-specific day applies to is to have recorded it. So the field exists **for that one purpose**.

The risk is not that somebody sets out to make pay depend on religion. It is that an attribute sitting on a person quietly acquires a second job — first a report groups by it, then something defaults from it, and eventually a wage depends on a field nobody thought was about wages. That is exactly how a shift came to be keyed to gender in the first version of this design, which is a mistake already paid for once here.

A day can apply to	And is decided by
Everybody	Nothing. Religion is never consulted.
One religion	The recorded value, and only where one is recorded
A named list of people	The list. Which is the honest shape whenever the arrangement is not really about a category at all.

Changing it: Record it, correct it or remove it at any time. Nothing already paid moves, because nothing already paid was ever decided by it.

This is checked rather than promised: a gate over the engine fails the build if the field is read anywhere that computes pay, hours, performance or permission. It was proven by making the pay calculation read it and watching the build go red.

Careful. Somebody whose religion nobody recorded, matched against a day scoped to a religion, **raises and is named.** Including them grants a paid day on an assumption; excluding them withholds one on the same assumption. Both are decisions about a real person that nobody actually made, and the second is the one they find out about on payday.

Done when: A religion-scoped holiday resolves for the people it applies to, raises rather than guessing for anybody whose religion is not on file, and no other calculation in the system can read the field at all.

5.9 · Give a weekly off to the people who have one, not to a category they belong to **IN THE APP**

An arrangement made with two people is not a company policy, and the moment it is stored as one it starts applying to the next person who happens to share their category. So it is a dated row against each person: how many Sundays a month they do not work, from when.

Two of the men here have two Sundays off a month, from a date in the middle of the year. Nobody else does — not the other men, not the other masters, not the people who joined later.

What is recorded	What is not
This person, this many Sundays, from this date	A rule about men, or about masters, or about seniority
Their stated monthly hours threshold, separately	A threshold worked out from the weekly off
That the two agree, checked by a test	One computed from the other

Changing it: Add, change or end somebody's weekly off from a date. Months already closed keep the arrangement that was in force when they were closed.

*The threshold and the weekly off are **two facts, both recorded, cross-checked and never derived from each other.** The two who have the weekly off also moved to a lower monthly threshold on the same date, and the arithmetic lines up exactly — which is worth checking and must not become a calculation. A system that recomputed the threshold would silently restate an already-paid month the next time somebody edited the shift table.*

Done when: A weekly off belongs to a named person from a named date, nobody acquires it by resembling them, and the stated threshold beside it was never computed from it.

5.10 · Pay somebody who was never employed, and let the payment be the whole record **IN THE APP**

Somebody comes for four days, is paid, and goes — the trial did not work out or the negotiation did not. There is no joining date, no leaving date and no salary, because none of those ever happened. A system that requires an

employment record before it will record a payment forces you to invent one, and an invented joining date is a fact about a real person that was never true.

So the payment stands alone. Nothing about it is derived, nothing calculates it, and nothing reports a missing salary — the amount handed over **is** the record.

Changing it: A trial who is taken on afterwards becomes a regular person from their real start date, and the trial days stay where they are rather than being rewritten into a spell.

A trial is never averaged into anybody's performance. Four days is not a person having a bad month, and scoring it as one puts something false on a record that follows them.

Careful. Attendance for somebody with no employment record and no payment is **refused, not paid zero**. It looks exactly like a trial and it is a hole: zero posts cleanly, reconciles, and is discovered by the person who was not paid. The system says what to record instead.

Done when: A payment can be recorded for somebody with no employment record at all, nothing raises a missing salary against them, the cost still lands in the right company and month, and attendance with no payment behind it is refused rather than valued at nothing.

5.7 · Mark IN and OUT as two messages, and let the pair make the day **ON A PHONE**

A day is not a tick. It is an **IN** and an **OUT**, and everything worth knowing is in the pair: how long somebody actually worked, whether they came late, whether they left early, whether a half-day was a half-day. One message a day cannot tell you any of that, and a supervisor reconstructing it at month end is guessing about somebody's wages.

What arrives	What the system does with it
IN , first of the day	Opens the day. Records the time it arrived, not the time it is read.
OUT , last of the day	Closes it. Hours worked is the difference, against that person's own shift.
A second IN with a day already open	Ignored as a repeat, and recorded as ignored. Nobody is punished for sending it twice.
OUT with no IN	Raised for a supervisor. It is not half a day and it is not a full one — it is unknown.
IN with no OUT by the cut-off	Raised the same way. The day stays open and unpaid until a person decides.

Changing it: The cut-off, the wording the phone accepts, and who reviews the exceptions are all settings, changed from a date.

The cut-off is a setting, not a constant, because a business that runs a late shift needs a different one and should not need a developer to say so.

Careful. An unmatched IN or OUT is **never** completed by assuming the shift. Filling in the missing half from the timetable produces a number that looks measured and was invented, and it is invented in the direction

of whoever wrote the code.

Done when: Every worked day is a matched IN and OUT with real times, every unmatched one is on somebody's list, and no missing half was ever filled in from the timetable.

5.8 · Close the day with what was actually finished, from the person who finished it **ON A PHONE**

Attendance says somebody was here. It does not say what came out. The end-of-day update is the second half — what was completed, against which design, at which stage — sent by the person who did it, on the day they did it.

This is the only moment that number is cheap to collect. On the 30th it is a memory, and a memory of the 4th is not evidence. Everything downstream — cost per piece, utilisation, what a unit is owed, where the work is stuck — is built on this one message, and none of it exists without it.

The update carries	Because
Which design	The cost has to land on something
Which stage or component	A top finished is not a set finished
How many	The quantity is what is paid for
Anything rejected, and why	A reject counted as output is a cost nobody sees

Changing it: What an update must carry, and by when, are settings. A business that adds a stage adds it here, not in a release.

*A day with attendance and no update is not zero output — it is an **unreported** day, and it says so. Those two are different facts and a system that shows them as the same number is lying quietly.*

Done when: Output is recorded on the day it happened by the person who did it, rejects are separated from output, and a day with no update reports as unreported rather than as nothing made.

Part 6 · Buying, checking, storing, sending

The chain from raw material to a parcel at somebody's door. Each step hands to the next, and none of them requires anyone to re-type what the previous one already knows.

Some of the making is sent out rather than done in-house, and for each of those jobs you have already decided who does it first.

Service sent out	Who does it first
Embroidery	Queen Worth
Digital Print	VD Enterprises
Foil Print	Lambodhar Print
Hand Dyeing	Hasan Bhai
Handwork	Rajesh Khan
Full Stitching	Cotton Sudaah
Partial Stitching	Aarya Trendz

Closed today, not closed forever. The list is closed today, not closed forever. A tenant adds an entry through core/tenant.js like any other change: effective-dated, append-only, and the closed months keep the list that applied then. What 'locked' rules out is a value appearing in a record that is not on the list — a lead whose source is a free-text guess, or a service sent to a vendor nobody appointed.

6.1 • Buy against a requirement, never against a hunch **IN THE APP**

What to buy comes from what is selling and what is already committed. A purchase order that does not name what caused it is a quantity nobody can trace back, and it is how stock quietly builds up.

Done when: Every purchase order names the requirement that caused it.

6.2 • Match three documents before paying a supplier **IN THE APP**

What you ordered, what arrived, and what you were billed for. Paying on the bill alone means paying for what did not arrive, and it is the most common way money leaks out of a business this size.

Done when: No supplier payment is possible without the three agreeing, or a recorded approval of the difference.

6.3 • Record a quality check with the person who did it **IN THE APP**

An anonymous pass cannot be investigated when the complaints arrive.

Done when: Every check names its checker, and rejected pieces are tracked to rework or to write-off.

6.4 • Treat goods sitting in somebody else's warehouse as your stock, in a location **IN THE APP**

Stock you still own but that sits in a marketplace's warehouse is a location like any other. Anything else and it disappears from your books until it sells, which understates what you own and hides it from ageing.

Done when: Every location holding your goods appears in the stock figure, wherever it physically is.

6.5 • Finish a sale at delivery, not at dispatch **IN THE APP**

A sale is not done when it leaves. It is done when it arrives and the money is in. Cash collected on delivery is money owed to you by the courier until it is settled, and treating dispatch as completion overstates both revenue and cash.

Done when: Money collected on delivery is tracked as owed until the courier settles it.

6.6 · Work out what raw material to buy, and show the working IN THE APP

Step 6.1 says buy against a requirement rather than a hunch. This is the requirement.

Buying from a feeling produces stockouts and dead stock in the same season, usually of different materials. The calculation is not complicated and its value is that it can be **disagreed with** — a number with its working shown is one a person can argue about, and a number without one only gets overridden.

Term	What it is	Where it comes from
Committed	Material the confirmed orders will consume	Open orders × what each design uses
In hand	What is physically here, everywhere it is	Stock across every location, including goods at a job worker
On order	Already bought, not yet arrived	Purchase orders not yet received against
Safety	The cushion for this material	A setting per material, not one number for everything
Requirement	Committed + Safety – In hand – On order	If it is not positive, buy nothing

Changing it: Safety stock, lead time and reorder level are settings per material and per vendor, effective-dated. Changing one today does not restate what was bought last month.

When to order is a separate question from how much. Order by the day the material is needed **minus** that vendor's own lead time — which is why lead time sits on the vendor record and is measured rather than agreed. A requirement raised on the day it is needed is a requirement raised too late.

Careful. A design with no material consumption recorded contributes **nothing** to the requirement and is listed as such, rather than being treated as needing none. Those two look identical in a total and mean opposite things.

Done when: Every purchase suggestion shows the five figures it came from, a design with no recorded consumption is named rather than silently counted as zero, and the timing comes from that vendor's measured lead time.

Part 7 · Getting paid, and getting the books right

Marketplaces do not pay you what the invoice said. They deduct commission, fees and taxes, and they get it wrong often enough that checking has to be automatic. And every one of those movements has to reach the books by itself.

Every operational action posts to the books by itself, from the record that caused it. Nothing is re-keyed into accounts, because re-keying is where the two versions of the truth appear.

What each action posts

When this happens	What is debited	What is credited
A sale is invoiced	The customer, or the bank on a cash sale	Sales, and the output tax
Money is received	Bank	The customer
Stock leaves on a sale	Cost of goods sold	Stock in hand
A return is accepted	Sales returns, and the output tax reversed	The customer
Material is received	Stock in hand, and the input tax credit	The supplier
A supplier is paid	The supplier	Bank
Wages are posted	Wages, by category	Payable to the person or the unit
A payout is settled	Bank, plus the commission and fees	The amount expected from that channel

Every line above is one half of a pair, and neither half can exist without the other. **A voucher that does not balance is refused at the moment it is made** — never saved and reported on later.

Tax

Rate	Per item category, effective-dated. An invoice keeps the rate that applied on its own date
Split	Within the state and outside it are different, decided from the two places, never from a setting somebody chose
Input credit	Claimed on goods accepted , not on goods received. Rejected material never earns a credit
Deductions at source	Tracked per party and per section, with the certificate against the record
Returns	Generated from the vouchers, per registration — never typed from a summary

Period locks, and correcting a filed month

Once a period is filed it is **locked**. It cannot be edited, and that is deliberate: a closed month that can still change is a month you cannot rely on having filed correctly.

A correction is a **new entry that names what it corrects**, dated in an open period. So the original filing still reproduces exactly what was filed, the correction is visible as a correction, and anybody asking "why does this differ" gets an answer instead of a mystery.

One workbook per financial year. **20 sheets**: two Read Me sheets, the combined productivity overview, then nine for the making side and nine for the staff side.

The rule that governs every cell in it

Every derived cell is a live formula referencing the sheets beside it — never a typed-in number. A total is not written as the number the system worked out. It is written as the calculation that produces it from the rows next

to it, so you can click any figure and see it being made.

A figure only the system can produce is a figure nobody can audit.

What *is* written as a plain value: the source facts. An attendance mark, a salary from the log, a quantity made, a rate. Everything derived from them is a formula.

And the formulas are checked by something that is not the system

A spreadsheet file can be written full of formulas that turn out to be broken, and it looks finished until somebody opens it. So after building, the workbook is opened in a **separate spreadsheet program**, every formula recalculated, and the result read back and compared against what the system itself calculated.

Two independent answers to the same question. If they differ, the workbook does not ship.

A missing side is said plainly, never filled in

A missing side is not an error. If only one side of the data was provided, the workbook is built from what exists, the other side's sheets are skipped, and it **says so plainly** — rather than fabricating the missing side so the file looks complete.

7.1 • Expect a specific amount for every order, before any payout arrives **IN THE APP**

Without an expectation there is nothing to compare a payout against, and a short payment looks identical to a correct one. The expectation is calculated from your own records at the moment of sale.

Done when: Every order carries what you expect to receive and when.

7.2 • Match every payout line to its order, and name every difference **IN THE APP**

A difference is either a legitimate deduction, or money you are owed. Both need naming. Netting the whole payout to one number makes the second kind invisible, and it is invisible money you never get back.

Done when: Every payout line is matched, every difference is named, and anything owed becomes a claim with evidence attached.

7.3 • Post everything to the books automatically, from the transaction **IN THE APP**

Re-keying into accounts is where the two versions of the truth appear. Every sale, purchase, payout and wage posts from the record that caused it, so the books and the operations can never disagree.

Done when: Every figure in the books can be clicked back to the transaction that created it.

7.4 • Lock a period once it is filed, and correct it only by a recorded entry **IN THE APP**

A closed month that can still change is a month you cannot rely on having filed correctly. Corrections are made as new entries that say what they correct, never by editing history.

Done when: A filed period cannot be edited, and every correction to it is a visible, dated entry.

7.5 • Register an invoice with the portal before it is a valid invoice, above the threshold **IN THE APP**

Over a turnover threshold, an invoice is not a document you issue — it is a document the government registers. It is sent to the portal, comes back with a reference number and a code printed on the face of it, and **an invoice above the threshold without one is not valid**, however correct its arithmetic.

That changes the order of operations, which is the part that catches people out: registration happens before the customer gets the invoice, not after.

Step	What has to be true
Invoice raised	Every field the portal requires is present — a missing one fails there, not here
Sent to the portal	Automatically, on issue, not as a batch somebody remembers
Reference and code returned	Both stored against the invoice and printed on it
Cancelled	Only within the window the portal allows; after that it is a credit note
Portal unreachable	Queued and retried, and the invoice is held rather than sent out unregistered

Changing it: The threshold, and whether it applies to a company, are settings with dates. Nothing already issued moves when they change.

The transport document is a separate registration with its own validity and its own cancellation window, and it is generated from the same invoice rather than re-entered. Re-entry is where the two stop agreeing about what is on the lorry.

Careful. Whether you are above the threshold is a **date-effective fact about your turnover**, not a permanent setting. A business that crosses it mid-year starts registering from that date, and the invoices before it stay valid exactly as issued.

Done when: Above the threshold, no invoice reaches a customer without its reference and code, an unreachable portal holds the invoice rather than releasing it unregistered, and the transport document is generated from the invoice rather than typed again.

Part 8 · What the system refuses to do

The half of the design that protects you. A feature list tells you what software does; this tells you what it will not do even when doing it would be convenient — and that is the half a business actually relies on.

Every one of these blocks the work rather than warning about it. A warning is something somebody clicks past at six in the evening.

16 checks, and every one of them blocks the work rather than warning about it.

The check	What it will not let through
logs resolve once	Every mandatory log gives exactly one row for every employed staff-month
components tie to design	Sum of a design's component values equals its raw recorded total
earnings tie to source	Karigar earnings equal the sum of the rows actually parsed
combined equals periods	The combined column is the sum of each period's columns, per unit
allocation ties to payroll	Allocated cost plus unallocated labour is the payroll, exactly
nothing dropped	Every source row is either matched or in Needs Review. None vanish
no formula errors	No error token and no broken reference anywhere in the output workbook
no person names in logic	The gate that keeps the engine data-independent
hours reference covers everyone	Every person's category resolves to an Hours Reference row
flat staff are flat	A Flat month earns exactly the salary in force — never more, never less
piece rate never uses salary	Piece-rate staff draw their rate from the work report, never Staff Master
reconciliation matches summary	FY earning in the reconciliation equals the sum of that person's months
roster is explained	Anyone Inactive but working, or working but not in Master, is listed
rows price themselves	Quantity times rate equals the value recorded on the row
bottleneck uses the set composition	No design's set count exceeds any slot the set actually requires
religion only decides holidays	RELIGION FOR HOLIDAY PURPOSE — checked, not promised

A single action must update every consequence of it, in one go, with nobody re-keying anything. If one of these needs a person to carry a number from one screen to another, it is not a system — it is a set of screens that happen to be next to each other.

1 · A sale, any channel

Do that one thing. Every item below must then be true without you touching it:

- stock deducted
- invoice with GST
- ledger (Dr Debtor/Bank, Cr Sales + Output GST)
- customer outstanding and lifecycle
- settlement expectation
- dashboard

2 · A marketplace order pull

Do that one thing. Every item below must then be true without you touching it:

- sales order created

- stock reserved
- pick list
- fulfilment
- on payout, reconciliation and variance check
- books

3 · A settlement import

Do that one thing. Every item below must then be true without you touching it:

- each line matched to its order
- commission, fees, TCS, TDS to books
- variance
- claim raised
- bank reconciliation
- SKU profit
- dashboard

4 · A return

Do that one thing. Every item below must then be true without you touching it:

- credit note
- refund
- a wrong return becomes dead stock, never restocked
- return cost to P&L
- return rate feeds design analytics

5 · A karigar production report

Do that one thing. Every item below must then be true without you touching it:

- pooled set completion and piece-rate earnings
- finished stock in
- payout in HR
- Karigar Wages posted
- cost per piece
- design profit

6 · A material purchase

Do that one thing. Every item below must then be true without you touching it:

- PO
- GRN
- three-way match

- raw stock in
- vendor payable
- input tax credit
- landed cost into COGS

7 · A staff attendance mark

Do that one thing. Every item below must then be true without you touching it:

- effective-dated salary resolved
- payroll
- Salaries posted
- productivity cost
- design cost

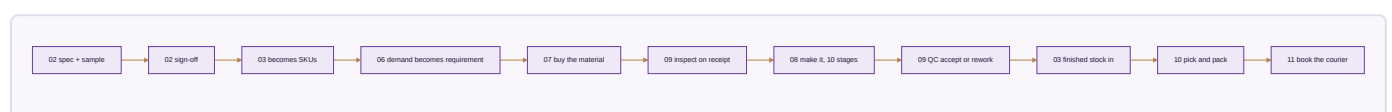
8 · A generated image or listing

Do that one thing. Every item below must then be true without you touching it:

- asset library
- product listing on the storefront and each marketplace
- marketing calendar
- published
- campaign return tracked back

A cascade proves one action fans out correctly. A flow proves the business can be **run**, start to finish. Each crosses many parts of the system, which is the point — the gaps between them are where systems usually fail.

Design to dispatch



The same chain, step by step:

1. 02 spec + sample
2. 02 sign-off
3. 03 becomes SKUs
4. 06 demand becomes requirement
5. 07 buy the material
6. 09 inspect on receipt
7. 08 make it, 10 stages
8. 09 QC accept or rework
9. 03 finished stock in

10. 10 pick and pack
11. 11 book the courier

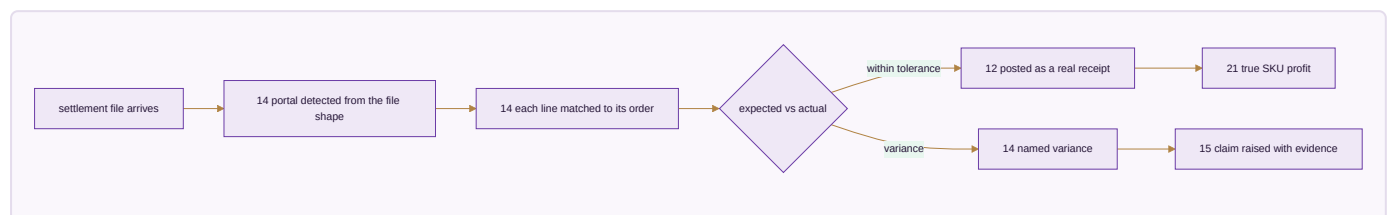
Order to cash



The same chain, step by step:

1. 05 / 15 order lands
2. 03 stock reserved
3. 12 invoice with GST
4. 10 picked and packed
5. 11 dispatched, AWB tracked
6. 11 delivered, COD collected
7. 12 receipt posted, invoice settled
8. 21 visible on the dashboard

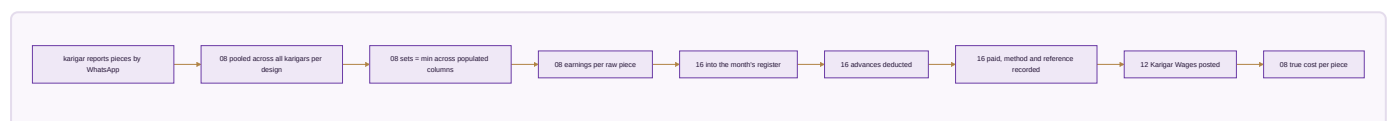
Settlement to books



The same chain, step by step:

1. settlement file arrives
2. 14 portal detected from the file shape
3. 14 each line matched to its order
4. 12 posted as a real receipt
5. 14 named variance
6. 15 claim raised with evidence
7. 21 true SKU profit

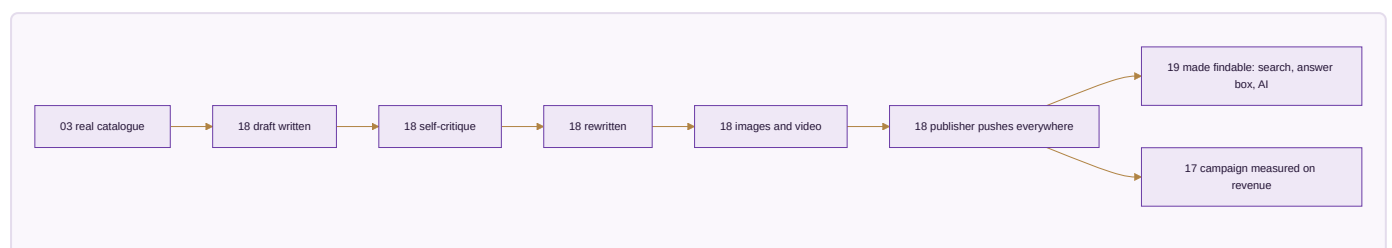
Karigar to payroll



The same chain, step by step:

1. karigar reports pieces by WhatsApp
2. 08 pooled across all karigars per design
3. 08 sets = min across populated columns
4. 08 earnings per raw piece
5. 16 into the month's register
6. 16 advances deducted
7. 16 paid, method and reference recorded
8. 12 Karigar Wages posted
9. 08 true cost per piece

Content to published



The same chain, step by step:

1. 03 real catalogue
2. 18 draft written
3. 18 self-critique
4. 18 rewritten
5. 18 images and video
6. 18 publisher pushes everywhere
7. 19 made findable: search, answer box, AI
8. 17 campaign measured on revenue

8.1 • Accept that a blocked run is the system working **WITH YOUR TEAM**

When a run stops because a rate is missing or a total does not tie, the instinct is to override it and move on. That instinct is what produces a payroll nobody can reconcile three months later. A blocked run is cheap; a wrong payment is not.

Done when: Everybody who runs payroll understands that a refusal names a real problem, and knows where to look.

8.2 • Keep the roster out of anything that gets shared **WITH YOUR TEAM**

Names, salaries and personal details belong in your system, behind permissions — not in a document, an export or a message. This guide itself contains no person's name for exactly that reason.

Done when: No exported document or shared report carries a name against a salary.

Part 9 · Changing anything, at any time

The promise this whole design is built to keep. **You can add, edit or remove anything, the moment you need to, and it takes effect at once.** No release, no developer, no ticket.

And the past does not move. Every change carries the date it starts from, so a month you already closed comes out the same tomorrow as it did yesterday.

effective date — *The date a change starts applying from. Records made before it keep the old value; records after it use the new one. Naya rate 1 tarikh se lagu. Purane mahine ka bill purane rate se hi banega — woh apne aap nahin badlega.* **audit trail** — *An automatic record of every change — what changed, who changed it, and when. Har entry ke saath naam aur time apne aap likha jaata hai. Baad mein koi bole "maine nahin kiya", toh register bata deta hai.*

People

What changes	Who can	Takes effect	Records already made
Somebody joins — a worker, a supervisor, an office staff member, a contractor	Admin, or an HR role	They exist from their start date and can be assigned work the same minute.	Nothing before their start date mentions them, because they were not there.
Somebody leaves, with or without notice	Admin, or an HR role	Marked as left from a date. Their sign-in stops, and no new work is assigned to them. Their record is kept, not deleted.	Every hour they worked, every piece they made and every rupee they were paid stays exactly as it was. Deleting the person would blank all of it and change months already closed — so the record remains and simply has an end date.
A replacement starts immediately, in the same position	Admin, or an HR role	Added with their own start date and given the position. Work in progress is reassigned to them from that date. No waiting, no release, no developer.	Work completed under the previous person stays credited to the previous person. Two people held the same position at different times, and every record knows which one applied when.
A pay rate, a piece rate or a salary changes	Admin, or an HR role	Applies from the date you set — which may be today, a future date, or a past one you are correcting.	Every completed period recalculates to the rate that applied then, not the new one. This is the single most important line in this register: a rate that silently applied backwards would change payments already made to real people.
What somebody is allowed to see and do	Admin	Takes effect on their next action. Screens they may no longer open stop opening.	Everything they did while they held the old permissions stays recorded, with the permissions they had at the time.

Structure

What changes	Who can	Takes effect	Records already made
A new company is opened, or an existing one is closed	Admin	It exists immediately with its own name, trading name, code and document numbering. The group view includes it from that date.	Group figures for earlier periods are unchanged, because the company did not exist in them.
A new way of selling is added — a marketplace, a shop, a counter, an export desk	Admin	Orders can arrive through it the same day. It appears in every report that breaks figures down by channel.	Earlier reports keep their own columns. A channel that did not exist then does not appear then.
A godown, a shop, a unit or a stock point is added, renamed or closed	Admin	Stock can move to and from it immediately.	Stock movements already recorded keep pointing at it, under the name it had at the time.
Which parts of the system this business uses at all	Admin	Turned on, a module appears in the menu with its screens ready. Turned off, it disappears from the menu. A steel plant, a clothing brand and a single creator each end up with a different system built from identical code.	Turning a module off hides its screens and keeps its records. Nothing is destroyed by tidying a menu.

Your words

What changes	Who can	Takes effect	Records already made
What the system calls things	Admin	Every screen, every report and every document changes wording at once. One business says order, another says job, another says matter, consignment, batch or booking — the record underneath is identical.	Documents already issued keep the wording they were issued with, because that is what the customer received.
Extra information you want to record that nobody else needs	Admin	Added to the screen immediately, with the type you choose — text, number, date, a list to pick from, a yes or no. Reportable from the moment it exists.	Older records simply have no value for it, which is the truth. They are never back-filled with a guess.
The steps your work moves through	Admin	Add a stage, rename one, reorder them or remove one. New work follows the new list from that moment.	Work already part-way through keeps the stage it is in, even if that stage has since been removed — a job does not teleport because somebody edited a list.
The layout and numbering of invoices, statements, labels and reports	Admin	The next document uses the new layout or the new numbering.	Documents already issued are never re-rendered. What the customer holds and what you hold stay identical.

Rules

What changes	Who can	Takes effect	Records already made
Turning a discretionary rule on or off	Admin	Applies to the next transaction. One business requires an approval below a price floor; another does not — same software, different setting.	Transactions already posted are not re-judged against a rule that did not apply to them.
Who has to approve what, and above which amount	Admin	The next request follows the new path.	Requests already approved keep the path they went through, and the names of who approved them.
Tax rates and the categories they attach to	Admin, or an accounts role	Applies from its effective date, which for tax is set by law rather than by you.	Every invoice keeps the rate that applied on its own date. A return filed for an earlier period recalculates to that period's rate — this is not a convenience, it is the only correct behaviour.
Which outside service is used for messages, payments, delivery or artificial intelligence	Admin	The next message, payment or shipment goes through the new one.	Everything already sent keeps the record of which service carried it, which is what you need when you query one.
The most the system may spend on paid outside services	Admin	Enforced immediately. Over the ceiling, the paid option is refused and the work completes on one that costs nothing.	Spending already recorded is unchanged.

What can never be switched off

Short on purpose. Every line is something a bank, an auditor, a customer or an employee relies on, and a setting that could remove it would remove their protection too.

Never changeable	Why
The audit trail	Who changed what, and when. A system where this can be switched off cannot be used to answer a dispute, so it cannot be switched off.
Every record naming the company it belongs to	Without it, figures from two companies merge and no report can be trusted again.
One business being unable to read another's records	This is not a preference. It is the promise that makes a shared platform usable at all.
Money kept as exact whole units	The alternative loses fractions of a rupee in ways nobody can trace afterwards.
Deleting nothing — records are ended, never erased	An erased record changes a period that was already closed, filed and possibly audited.
Never asking for a marketplace, bank or account password	The system connects through proper keys that you can withdraw. A password would hand over an account you cannot take back.

Every value that can change over time is kept as a log rather than as a single figure — a salary, a rate, a threshold, a role, a person's basis.

A value is never overwritten	The open entry is closed off and a new one is appended
History stays intact	Every earlier value is still there, with the dates it applied between
A future date is allowed	An entry dated ahead activates by itself when that month arrives
Superseded is not deleted	An entry that has been replaced stays readable, because a report for an earlier period still needs it
No match is an error	Never zero

That last line is worth reading twice. **Silently returning zero is how a person earns nothing without anyone noticing** — the run completes, the report looks normal, and somebody is not paid. So the system stops and names what it could not resolve, for whom, and for which month.

This is also how you set a change in advance. A rate agreed today and starting next month is entered today with next month's date, and it applies itself on the first — nobody has to remember.

9.1 • Work the case where somebody leaves without notice **IN THE APP**

This is not hypothetical and it is the reason the whole effective-dated design exists. A master leaves on the 14th with no notice. A replacement is found and starts on the 15th. Both are recorded the same morning.

Step by step:

1. Mark the person who left as having ended, with the 14th as the date. **Their record is kept, not deleted** — every hour they worked, every piece they made and every rupee they were paid stays exactly as it was.
2. Add the new person with the 15th as their start date, and give them the position.
3. Set their rate, from the 15th.
4. Reassign work in progress to them from the 15th.
5. That is the whole change, and it took a few minutes.

The question somebody asks later	The answer
Last month's payroll — did it move?	No. It resolves the rates and people that applied then.
Who made the pieces finished on the 12th?	The person who left. Credited to them, permanently.
Who made the pieces finished on the 16th?	The new person.
Two people held one position — is that a conflict?	No. They held it at different times, and every record knows which.
Can we delete the person who left?	No, and you would not want to — it would blank their history and change months already paid.

Done when: The change is made the same day, and a report for the previous month produces an identical figure before and after.

9.2 · Change a rate for a past period you are correcting **IN THE APP**

Sometimes a rate was recorded wrongly and the correction genuinely belongs in the past. That is allowed — you set the date it should have applied from. The difference is that the change is **deliberate and dated**, and everything affected is recalculated visibly rather than silently.

Done when: A backdated correction shows exactly which periods it changed and by how much, before you confirm it.

9.3 · Turn off the parts of the system you do not use **IN THE APP**

You do not need everything. Turning a module off removes it from the menu and keeps every record it ever held — tidying a menu never destroys data, and turning it back on later brings everything back exactly as it was.

Done when: The menu shows only what this business uses, and nothing was lost in making that true.

Part 10 · The rulebook that applies to you

Every rule states what happens **and what the system will never do instead**. The second half is the part worth reading — it is what you are relying on when you are not looking.

database — Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost. *Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.* **table** — One kind of information inside the database — all your customers in one, all your orders in another. *Almari ka ek khaana. Ek khaane mein sirf customers, doosre mein sirf orders.* **schema** — The written plan of what information the system keeps and how the pieces connect. *Makaan ka naksha. Deewar uthane se pehle kaagaz pe tay hota hai kaunsa kamra kahaan hai.* **migration** — A recorded change to the shape of the database, so every copy of the system can be updated the same way, in the same order. *Naksha badla toh likh ke rakha — taaki har site pe wahi badlav, usi tarike se ho.* **backup** — A copy of everything, kept somewhere else, so a mistake or a failure does not lose your work. *Zaroori kaagzaat ki photocopy, doosri jagah rakhi hui. Asli jal jaaye toh bhi kaam nahin rukta.* **API** — The agreed way two pieces of software talk to each other, so one can ask the other for something and get a predictable answer. *Waiter. Aap kitchen mein nahin jaate — waiter ko order dete ho, wahi khaana le aata hai. Waiter badal jaaye toh bhi order dene ka tarika wahi rehta hai.* **storage** — Where files are kept — photographs, invoices, scanned documents. Different from the database, which keeps information rather than files. *Almari ke bagal wala godown. Register almari mein, par bade dabbe aur photo godown mein.* **queue** — A waiting line for work that does not have to finish this second — sending a hundred messages, building a big report. *Darzi ki dukaan ka parchi system. Kaam parchi pe likh ke lag gaya line mein; customer khada intezaar nahin karta.* **model** — The piece of artificial intelligence that reads or writes text, tags a photograph, or answers a question. *Ek bahut padha-likha assistant. Kaam accha karta hai, par har baat pe usse poochho toh kharcha aur waqt dono lagta hai.* **provider** — A company whose service the system uses — for messages, for payments, for artificial intelligence, for delivery. *Supplier. Ek supplier maal na de toh doosre*

se le lo — kaam nahin rukna chahiye. **spend ceiling** — A maximum amount the system is allowed to spend on paid services, after which it refuses to spend more instead of warning you. *Jeb mein utne hi paise leke nikle jitna kharch karna hai. Khatam matlab khatam — udhaar nahin.*

285 rules across 22 modules. Every one says what happens *and* what the system will never do instead.

#	Module	Rules
01	Platform	25
02	Design & Sampling	7
03	Inventory & Catalog	14
04	CRM	9
05	Sales	18
06	Planning & Requirements (MRP)	8
07	Purchase	12
08	Manufacturing	20
09	Quality & Compliance	7
10	Warehouse	8
11	Logistics	11
12	Accounting & GST	24
13	Treasury & Financial Planning	8
14	Settlement	13
15	E-commerce / OMS	19
16	HR & Payroll	22
17	Marketing	10
18	AI Content Engine	11
19	SEO, AEO & AIO	6
20	Projects & Collaboration	9
21	Dashboard & BI	9
22	AI Assistant, Agents & Automation	15

Module 01 · Platform — 25 rules

R01.1 Every business record names the company it belongs to

- **When** any record is written — a sale, a movement, a voucher, an employee
- **Then** its company is stored on the row itself
- **Never** inferring the company from who happens to be logged in, which silently mis-files every record made by someone who works across two of them

R01.2 One company cannot read another company's records

- **When** a query runs for a user scoped to one company
- **Then** rows belonging to any other company are not returned at all
- **Never** filtering in the screen while the data is already loaded — a filter can be removed, a scope cannot

R01.3 The audit trail has no off switch

- **When** anything touching money, stock, price, tax, pay or master data changes
- **Then** the change and its before-image are written in the same transaction as the change itself
- **Never** allowing a setting, a role or a migration to disable it — either both land or neither does

R01.4 An update records what it was, not only what it became

- **When** a row is changed
- **Then** the current row is read first so the before-image is what was really there
- **Never** trusting the caller's idea of the old value, which makes the trail a record of intentions rather than of facts

R01.5 A table nobody thought to audit is refused

- **When** code writes to a table that is not on the audited list
- **Then** the write is refused and the table is named
- **Never** letting it through quietly, which is how a money column ends up outside the trail without anyone deciding that

R01.6 Deletion is a reversal, never a removal

- **When** a user deletes anything
- **Then** the record is voided, marked, and still readable with the reason
- **Never** removing the row — eight years of trail cannot survive a DELETE

R01.7 A module that is not in the canonical list cannot join the bus

- **When** code subscribes to a business event
- **Then** the module number is checked against modules.js and refused if unknown
- **Never** letting an unregistered listener attach, which is how a cascade gains a step nobody can find later

R01.8 A cascade is all of it or none of it

- **When** one business event fans out to stock, ledger, customer and documents
- **Then** every step commits together, or the whole thing is rolled back
- **Never** leaving stock moved and the ledger unposted, which is the exact state no report can ever explain

R01.9 A handler that throws takes the transaction with it

- **When** any subscriber to an event fails
- **Then** the emitting transaction fails too
- **Never** swallowing the error so the originating action appears to have succeeded

R01.10 No capability depends on a single outside service

- **When** any capability is used — books, courier, payments, AI, storage, GST
- **Then** an ordered list of interchangeable providers is tried, ending on one that needs nothing connected
- **Never** having one provider whose outage stops the work, however good that provider is

R01.11 A failing provider is taken out of the list, not hammered

- **When** a provider fails repeatedly
- **Then** it is tripped open, skipped entirely, and retried once after a cooldown
- **Never** retrying into a dead service on every call while a working alternative sits further down the list

R01.12 A spend ceiling refuses, it does not warn

- **When** a paid call would take spending past the ceiling set for it
- **Then** that provider is refused and the work completes on a free one
- **Never** letting it through with a warning nobody reads, and never refusing only the first provider while the same spend reroutes to the next

R01.13 The system never asks for a marketplace, bank or account password

- **When** any integration is connected, by any module, including a chatbot or an agent
- **Then** a scoped, revocable key is requested instead, cancellable from the provider's side without changing the login
- **Never** accepting, storing, echoing or transmitting an account password — there is no screen, no import and no support flow that takes one

R01.14 Card and bank credentials never reach application code

- **When** a payment needs a card or bank detail
- **Then** the provider's own secured field takes it directly
- **Never** passing it through this system, even in transit, even unlogged — what is never received cannot be leaked

R01.15 Consent and retention are two different clocks

- **When** a person's data is held
- **Then** why it may be used and how long it is kept are tracked separately, and an erasure request is resolved against both
- **Never** treating a legal retention period as consent to keep using the data for anything else

R01.16 A scoped key is revocable without touching the login

- **When** an outside service is connected

- **Then** a key limited to what that capability needs is stored, and the connection records which capability it serves
- **Never** storing a credential that can do more than the capability requires, because the day it leaks is the day that difference matters

R01.17 A webhook is verified, idempotent and never silently dropped

- **When** a payment, courier, storefront or messaging provider calls in
- **Then** the signature is checked, the external id makes a repeat delivery a no-op, and a failure is logged with its payload for retry
- **Never** trusting an unsigned call, and never processing the same external id twice — a duplicated payout or a duplicated order is indistinguishable from a real one afterwards

R01.18 A trade is added as data, never as a version of the software

- **When** a business in a trade the system has never seen signs up
- **Then** its vocabulary, stages, extra fields, documents, rule switches and starting reference data arrive as one configuration file, and every screen reads back in that trade's words
- **Never** a branch, a fork or a bespoke build per industry — that is a consultancy with software attached, and it is the thing that stops a product from being one

R01.19 A pack is data and can never be code

- **When** a pack is loaded from any source
- **Then** every value in it is inspected, at any depth, and a function anywhere inside it refuses the whole pack
- **Never** letting configuration carry behaviour — the moment a pack can run code, adding a trade is a code change again and the guarantee in R01.18 is worthless

R01.20 A pack may rename a concept, never invent one

- **When** a pack declares its vocabulary
- **Then** each entry is matched against the fixed list of concepts the engine has, and an unknown one refuses the pack
- **Never** accepting an unrecognised word as a new concept, which turns "vocabulary" into a place to put anything and leaves the screens with a name for something that does not exist

R01.21 A pack extends tables that exist, and nothing else

- **When** a pack adds fields
- **Then** the table is checked against the real schema and the field type against the types the engine can store
- **Never** creating a table on a customer's behalf from a configuration file, which puts the shape of the database outside the reach of the schema test that guards it

R01.22 Money in a pack is money everywhere else

- **When** a pack adds a field whose name reads as an amount, a price, a cost, a total, a fee or a rate
- **Then** it must be declared in paise, and a plain number refuses the pack

- **Never** letting a trade introduce a floating-point rupee through the side door after the whole schema was built to keep them out

R01.23 No pack can switch off a guarantee

- **When** a pack sets a rule off
- **Then** the rule id is checked against the rulebook, and against the list of rules no pack may touch — company scoping, the audit trail, the posting rules, group elimination and roster privacy
- **Never** a trade opting out of the things that make the books trustworthy; it may call an invoice whatever it likes and may not decide its trail is optional

R01.24 A rule a pack never mentions is on

- **When** a rule is looked up for a trade
- **Then** the rulebook is the default and the pack is read as an exception list — silence means the rule applies
- **Never** treating the pack as a permission list, which would mean every rule added after a pack was written silently applies to nobody who is using it

R01.25 An invalid pack is refused whole, never half-loaded

- **When** a pack fails any check
- **Then** every problem in it is reported at once and none of it is applied
- **Never** partially loading a trade, which leaves a system whose vocabulary and rules disagree with each other and no way to tell which half is live

Module 02 · Design & Sampling — 7 rules

R02.1 A style becomes a SKU only after sign-off

- **When** someone tries to create a catalogue record for a design
- **Then** the design must already have passed sample sign-off
- **Never** letting a SKU exist for something with no agreed specification, which puts an unmakeable item on sale

R02.2 Every version of a specification is kept

- **When** a sample round changes a measurement, a fabric or a trim
- **Then** a new version is written and the old one stays readable
- **Never** editing the specification in place — a karigar paid against last month's spec must still be able to show what it said

R02.3 A costed trial carries the date its rates came from

- **When** a sample is costed
- **Then** the rate and the date it was in force are both stored on the trial
- **Never** recosting an old trial with today's rates and presenting the result as what it cost then

R02.4 A design with no ownership record is flagged, not blocked

- **When** a design reaches sign-off with no trademark or copyright status on file

- **Then** it proceeds and is listed as unprotected
- **Never** silently treating it as protected, which is only discovered when a near-identical listing appears and there is nothing to act on

R02.5 The first-shown date is recorded when it happens

- **When** a design is first shown publicly — an exhibition, a listing, a lookbook
- **Then** that date is stamped and never editable afterwards
- **Never** backdating it later, which is precisely the field a dispute turns on

R02.6 A rejected sample keeps its reason

- **When** a sample round is rejected
- **Then** the reason is recorded against the version
- **Never** closing it with a status alone, which loses the only information that stops the same mistake in the next round

R02.7 A specification cannot be deleted while stock exists against it

- **When** someone removes a design that has ever been made
- **Then** it is archived and stays linked to every piece produced from it
- **Never** orphaning finished stock from the specification it was made to

Module 03 · Inventory & Catalog — 14 rules

R03.1 Stock is one number per SKU, per location, per stage

- **When** any module asks how much there is
- **Then** it reads the one quantity, with the channel recorded on the movement rather than on the stock
- **Never** keeping a separate stock figure per channel — the last piece sold on one marketplace has to vanish from the other ten at the same instant, which per-channel inventory cannot do

R03.2 Negative stock is a fault, not a state

- **When** an issue would take a quantity below zero
- **Then** the issue is refused
- **Never** recording a negative balance and leaving someone to explain it at month-end

R03.3 Selling a kit decrements every component

- **When** a kit or combo SKU is sold
- **Then** each component SKU is decremented at order time
- **Never** decrementing only the kit, which leaves the components sellable twice

R03.4 A kit with no components is refused

- **When** an item is marked a kit but lists nothing
- **Then** the record is refused and named

- **Never** accepting it and silently decrementing nothing on every sale

R03.5 Stock value ties to the item cost, always

- **When** stock is valued
- **Then** the value is computed from the quantity and the item cost
- **Never** storing a valuation that can drift from the quantity it is supposed to describe

R03.6 Every movement has a source, a destination, or both

- **When** a stock movement is recorded
- **Then** at least one end is named
- **Never** accepting a movement from nowhere to nowhere, which is how quantity appears without a cause

R03.7 A quantity is a whole number above zero

- **When** a movement is written
- **Then** a non-integer or non-positive quantity is refused
- **Never** accepting a negative movement as a shorthand for a reversal — a reversal is its own movement with its own reason

R03.8 Goods in someone else's warehouse are still yours

- **When** stock sits in a channel's own warehouse under consignment or sale-or-return
- **Then** that warehouse is a location like any other and the stock is counted, valued and aged there
- **Never** letting it drop off the books until it sells, which understates both stock and exposure

R03.9 Fabric in metres and pieces in numbers share one item master

- **When** an item is defined
- **Then** its unit of measure is a property of the item
- **Never** building a second item master for a second unit, which splits the one stock number this module exists to protect

R03.10 A listing needs the packed size and weight before it can go out

- **When** a product is pushed to a channel
- **Then** packed dimensions and weight must be present
- **Never** listing without them, because that is the field every courier weight dispute is settled on

R03.11 The channel's own code for a product is mapped, not assumed

- **When** a product exists on a marketplace
- **Then** that channel's identifier is stored against ours
- **Never** matching on name or on a code we invented, which mis-posts every settlement line for that product

R03.12 A duplicate master record is merged, never left as two

- **When** the same customer, vendor or design is detected twice

- **Then** they are merged and both old identifiers keep resolving
- **Never** leaving two live records, which splits every total that record appears in

R03.13 A price is per channel and dated

- **When** a channel price is set
- **Then** it is stored against that channel with the date it takes effect
- **Never** holding one price and reading it as if it applied everywhere and always

R03.14 Dead stock is named as dead stock

- **When** an item has not moved for the period set for it
- **Then** it appears on the dead-stock register with its age and carrying value
- **Never** leaving it inside the general stock figure where it reads as healthy inventory

Module 04 · CRM — 9 rules

R04.1 One customer, one record, whichever channel they arrived by

- **When** the same person orders on a marketplace and later at the counter
- **Then** both land on one record with the channel noted on each order
- **Never** creating a second customer per channel, which makes lifetime value meaningless

R04.2 A document is filed against the record it belongs to

- **When** any agreement, receipt, certificate or scan is stored
- **Then** it is attached to the order, party, case or employee it concerns
- **Never** filing it in a folder that has to be remembered rather than found

R04.3 A signed copy files itself back

- **When** a document sent for signature is signed
- **Then** the signed version returns to the same record automatically
- **Never** leaving the signed copy in an inbox while the record still shows it as pending

R04.4 A ticket carries the order it is about

- **When** a question arrives by chat, email or phone
- **Then** it is tied to the order or account it concerns, with the history already on screen
- **Never** opening a ticket with no link, which makes the first reply a request to explain again

R04.5 Feedback attaches to the item, not only the buyer

- **When** a rating or complaint arrives after delivery
- **Then** it is attached to the design or item it is actually about
- **Never** holding it only against the customer, which hides a complaint-prone item as a scatter of unrelated gripes

R04.6 A customer's consent travels with their data

- **When** a customer record is used for marketing or profiling
- **Then** the consent captured at the point it was given is checked first
- **Never** assuming that having the data implies permission to use it for anything

R04.7 A merged customer keeps both histories

- **When** two customer records are merged
- **Then** every order, ticket and document from both survives on the surviving record
- **Never** discarding the shorter history to make the merge simple

R04.8 Credit state is read at the moment of the order

- **When** a B2B order is placed
- **Then** the customer's outstanding and limit are evaluated then
- **Never** using a figure cached from the last sync, which is how a party goes past its limit between refreshes

R04.9 A closed ticket keeps what resolved it

- **When** a ticket is closed
- **Then** the resolution is recorded on it
- **Never** closing with a status alone, which loses the answer the next identical question needs

Module 05 · Sales — 18 rules

R05.1 Every sale carries its company and its channel

- **When** an order is created on any channel
- **Then** both are written on the order
- **Never** leaving either to be inferred later from the document number or the warehouse

R05.2 A sale posts stock and ledger together

- **When** a sale is confirmed
- **Then** stock is deducted, the invoice is raised, and the ledger is posted in one transaction
- **Never** invoicing without moving stock, or moving stock without posting

R05.3 If the ledger refuses, the stock never moved

- **When** the posting half of a sale fails
- **Then** the stock movement is rolled back with it
- **Never** leaving the goods gone and the books untouched

R05.4 A quote becomes an order without being retyped

- **When** a quotation is accepted
- **Then** the order is created from it, carrying the same lines and prices
- **Never** re-entering the lines, which is where the price on the quote and the price on the invoice start to differ

R05.5 A price below the floor needs an approval, not a note

- **When** a line is priced under the floor set for it
- **Then** the order waits in the approvals queue with the rule that stopped it named
- **Never** letting it through with a comment box, which is a discount policy nobody can enforce

R05.6 An export invoice knows it is an export

- **When** an order ships outside the country
- **Then** the LUT or IGST treatment, currency and shipping terms are set on the order itself
- **Never** treating it as a domestic invoice and correcting the tax afterwards

R05.7 A counter sale is the same order record

- **When** someone buys at the counter
- **Then** the same order table records it, with the counter as the channel
- **Never** running the till on a separate book that has to be merged later

R05.8 A credit sale reserves the credit at the moment it is taken

- **When** a B2B order is accepted on credit
- **Then** the exposure is committed against the party immediately
- **Never** counting it only when the invoice is raised, which lets several orders each fit inside the same limit

R05.9 A dispatch cannot exceed what was ordered

- **When** a shipment is prepared
- **Then** quantities are checked against the order line
- **Never** shipping over, which becomes an invoice the customer never agreed to

R05.10 A cancelled order releases what it held

- **When** an order is cancelled
- **Then** reserved stock and committed credit are both released
- **Never** leaving stock reserved against a dead order, which shows the business as out of goods it actually has

R05.11 An AWB belongs to the shipment, not the courier integration

- **When** a tracking number is recorded, typed in or fetched
- **Then** it is stored on the shipment
- **Never** making the number reachable only through whichever courier API produced it, which loses it the day that courier is dropped

R05.12 A subscription renewal is a new order

- **When** a subscription renews
- **Then** a fresh order is created with its own stock, invoice and posting
- **Never** extending the original order, which makes the revenue of two periods indistinguishable

R05.13 A sale to a sister company is marked as one

- **When** the counterparty is another company in the group
- **Then** the counterparty company is recorded on the entry
- **Never** posting it as an ordinary outside sale, which inflates the group turnover by trade it never did

R05.14 A quote or proforma number carries its type and financial year

- **When** a quotation or proforma is raised
- **Then** it is numbered Q-{FY}-#### or PI-{FY}-####, sequential within that company and year
- **Never** sharing one sequence between quotations and proformas, which makes a proforma indistinguishable from a quote in the register

R05.15 A quote line with no description, no quantity or a negative rate is not a line

- **When** a quotation is totalled
- **Then** only lines with a description, a quantity above zero and a rate of zero or more are counted
- **Never** letting a half-filled row contribute a number to the total

R05.16 An export line carries no GST

- **When** a quotation or invoice is marked export under LUT
- **Then** the GST percentage is zero and the document says why
- **Never** applying the domestic rate and correcting it after the buyer queries the total

R05.17 A made-to-measure order has two money legs, and both are visible

- **When** a customisation order is accepted
- **Then** the advance and the balance are recorded as separate amounts with their own dates, and the balance stays owed until dispatch
- **Never** showing one payment at the end, which hides money already taken and work already owed

R05.18 A customisation quote keeps every round of the negotiation

- **When** a price is revised during a bespoke enquiry
- **Then** each quoted figure is kept in order with what changed
- **Never** overwriting the earlier figure, which is the one the customer remembers agreeing to

Module 06 · Planning & Requirements (MRP) — 8 rules

R06.1 A forecast is labelled a forecast wherever it appears

- **When** a projected figure is shown beside actuals
- **Then** it is visually and structurally distinct
- **Never** letting a forecast total sit in the same column as a real one, which is how a plan becomes a reported result

R06.2 A requirement run reads live stock, not a snapshot

- **When** the MRP run explodes requirements

- **Then** it reads the current quantity at the moment it runs
- **Never** planning against a nightly copy, which orders material the business already has

R06.3 A requirement names what caused it

- **When** the run produces a shortfall
- **Then** the order, forecast or reorder level that generated it is recorded on the line
- **Never** producing a bare quantity nobody can trace back to a demand

R06.4 Stock already on order counts against the shortfall

- **When** the run computes what to buy
- **Then** open purchase orders are netted off first
- **Never** ignoring them and ordering the same material twice

R06.5 A budget ceiling refuses, it does not warn

- **When** a proposed purchase would exceed the open-to-buy ceiling
- **Then** it is held for approval with the ceiling named
- **Never** raising it with a warning, which makes the ceiling advisory and therefore not a ceiling

R06.6 A lead time is per vendor and per item

- **When** a run works out when to order
- **Then** it uses the lead time recorded for that vendor and that item
- **Never** applying one global lead time, which under-orders the slow lines and over-orders the fast ones

R06.7 A run is kept, not overwritten

- **When** the MRP run executes again
- **Then** the previous run stays readable with its inputs
- **Never** replacing it, which makes it impossible to see why last week's decision was taken

R06.8 A seasonal signal cannot silently become a permanent one

- **When** a festival or season inflates demand
- **Then** the period it applies to is stored with the signal
- **Never** folding a spike into the baseline, which keeps ordering for a festival all year

Module 07 · Purchase — 12 rules

R07.1 Nothing is paid without a three-way match

- **When** a vendor invoice is approved
- **Then** the purchase order, the goods received note and the invoice must agree
- **Never** paying on the invoice alone, which pays for goods that never arrived

R07.2 A short or damaged receipt is recorded as received short

- **When** the GRN quantity is below the PO quantity
- **Then** the difference is recorded with its reason and the payable follows the received quantity
- **Never** receiving the full quantity to make the match pass

R07.3 Input tax credit is claimed against a real document

- **When** ITC is taken on a purchase
- **Then** the vendor invoice and its tax detail are on file
- **Never** claiming credit from a payment record alone, which is the claim that fails reconciliation

R07.4 Landed cost reaches the item, not just the P&L

- **When** freight, duty or insurance is attached to a purchase
- **Then** it is apportioned into the cost of the items received
- **Never** expensing it separately, which understates the cost of every piece made from that material

R07.5 A vendor price is dated

- **When** a rate is agreed with a supplier
- **Then** it is stored with the date it takes effect
- **Never** overwriting the old rate, which makes last month's purchase look mispriced

R07.6 A purchase order over its approval level waits

- **When** a PO exceeds the value a role may approve
- **Then** it goes to the approvals queue naming the rule and the level
- **Never** splitting it into smaller orders to fit under the limit — the split is detected and the parts are assessed together

R07.7 A vendor with no active record cannot be paid

- **When** a payment is raised
- **Then** the vendor must exist, be active, and have its bank detail verified
- **Never** paying to detail typed onto the payment itself, which is the single most common route for payment fraud

R07.8 A change to vendor bank detail is treated as high risk

- **When** a vendor's bank account is changed
- **Then** the change is approved by a second person and the old detail is kept
- **Never** accepting a change from an email instruction alone

R07.9 A job-work despatch stays on the books

- **When** material is sent to a contractor
- **Then** it moves to a job-work location and remains this company's stock
- **Never** writing it out on despatch, which loses material the business still owns

R07.10 An insurance policy is linked to what it covers

- **When** a policy is recorded
- **Then** the stock, premises or shipment it covers is named on it
- **Never** holding policies as documents with no link, which is discovered only at the moment of a claim

R07.11 The three-way match is arithmetic, not a judgement

- **When** a vendor invoice is checked
- **Then** the payable equals the received quantity × the purchase-order rate, and the purchase order, the goods receipt and the invoice must all agree on quantity and value
- **Never** passing an invoice whose value exceeds received quantity × agreed rate, and never letting an override happen without recording who made it and why

R07.12 A material is sourced down a ranked list, not from whoever answers

- **When** a material has to be bought
- **Then** the vendors ranked for that material are approached in their priority order
- **Never** defaulting to the last vendor used, which is how a price rise becomes permanent without anyone deciding

Module 08 · Manufacturing — 20 rules

R08.1 Sets are pooled across every karigar before the minimum is taken

- **When** completed sets are counted for a design
- **Then** every karigar's pieces for that design are pooled first, and the set count is the minimum across the populated member columns of the pool
- **Never** counting sets per karigar row and adding them up, which loses every set completed by two people between them

R08.2 A surplus piece is paid for, and is not a set

- **When** a karigar makes more of one garment than the set needs
- **Then** the extra is named individually and paid at its own piece rate
- **Never** adding it to the set count, and never leaving it unpaid because it did not complete a set — the person made it either way

R08.3 A design counts on the garments it actually has

- **When** a design is made of fewer garment types than the usual set
- **Then** it is counted on the members it does have
- **Never** returning zero because an optional member is absent, which silently unpays a whole design

R08.4 A missing rate posts zero and is flagged, never guessed

- **When** a design has no entry in the stitching rate master
- **Then** it costs zero and the design is named in the summary
- **Never** inferring a rate from a similar design — a guessed rate is a wrong payment to a real person

R08.5 A two-row heading is read as two rows

- **When** the production grid uses a merged heading over garment columns
- **Then** both header rows are read so repeated garment names stay distinct columns
- **Never** reading only the first row, which collapses three Dupatta columns into one and undercounts the work

R08.6 A karigar written as a pair stays one unit

- **When** two names share one row as a working pair
- **Then** they are treated as a single paying unit
- **Never** splitting them into two karigars, which halves each person's recorded output and breaks the payout

R08.7 Several years of grids pool into one set of figures

- **When** more than one production workbook is supplied
- **Then** their grids pool into a single costing
- **Never** reporting each file separately, which double-counts nothing but hides the sets completed across a year boundary

R08.8 Cost per piece is independent of set completion

- **When** the cost of a design is worked out
- **Then** each raw piece is costed at its own rate
- **Never** costing by completed sets, which values an unfinished set at nothing while the labour has already been spent

R08.9 A production report moves stock and pay together

- **When** a karigar production report is accepted
- **Then** finished stock comes in, the payout is raised in HR, wages post to the ledger, and the design cost updates — in one transaction
- **Never** taking the stock in and settling the pay in a separate pass, which is how the two disagree

R08.10 Material issued to production leaves raw stock at the moment it is issued

- **When** a production order consumes material
- **Then** raw stock is reduced and work in progress increases
- **Never** consuming at completion, which shows material as available while it is already cut

R08.11 A bill of materials is versioned with the design

- **When** a production order is created
- **Then** it captures the BOM version in force at that moment
- **Never** reading the current BOM when costing an old order, which recosts history

R08.12 Wastage is recorded, not absorbed

- **When** consumption exceeds the BOM
- **Then** the excess is recorded as wastage against the order with its reason

- **Never** quietly increasing the BOM to match what was used, which destroys the only signal that something is going wrong

R08.13 A stage cannot be skipped without being recorded as skipped

- **When** work moves past a defined stage without that stage being marked
- **Then** the skip is recorded on the order
- **Never** letting the stage silently complete, which makes every stage-time figure fiction

R08.14 An advance to a karigar is a balance, not a deduction from nowhere

- **When** an advance is paid
- **Then** it is held against that karigar and recovered from later payouts, with the running balance visible
- **Never** deducting an amount at payout time that cannot be traced to a specific advance

R08.15 A rework carries the cost of the rework

- **When** a piece is returned to a stage to be redone
- **Then** the additional labour is costed to the design that caused it
- **Never** costing it as new production, which makes a failing design look as profitable as a good one

R08.16 Material consumed is the average per piece times the pieces made

- **When** consumption is costed against a production run
- **Then** consumption equals the average consumption per piece $\times$ pieces produced, and the difference against the bill of materials is recorded as wastage
- **Never** back-fitting the average to whatever was issued, which makes wastage mathematically impossible to see

R08.17 A set type comes from the rate master, and an inferred one says so

- **When** a design is classified into a set type
- **Then** the rate master's Set column decides it; when the design is absent, the type is inferred from which garment columns actually carry pieces and the design is flagged as inferred
- **Never** presenting an inferred classification as though it came from the master

R08.18 An alteration caused by the karigar's own mistake is unpaid

- **When** a piece is reworked because of an error by the person who made it
- **Then** the alteration hours are recorded and paid at zero
- **Never** paying for the rework at the standard alteration rate, and never leaving the hours unrecorded — the time still happened and the design still bore the cost

R08.19 Alteration time is paid at the alteration rate, not the piece rate

- **When** admin-assigned alteration hours are settled
- **Then** they are paid at the hourly alteration rate in force and added to that karigar's payout
- **Never** folding alteration hours into the piece count, which corrupts both the production figure and the earnings figure at once

R08.20 A contract worker paid by the hour has no attendance row

- **When** a contract role is settled
- **Then** payment is hours worked × the agreed hourly rate, recorded against the person without an attendance record
- **Never** forcing a contract worker through the salaried attendance model, which produces a monthly figure nobody agreed to

Module 09 · Quality & Compliance — 7 rules

R09.1 A failed check blocks the next stage

- **When** an inspection fails
- **Then** the batch cannot progress until it is passed, reworked or written off
- **Never** letting it move with the failure noted, which sends a known defect to a customer

R09.2 A check names the person who did it

- **When** any inspection is recorded
- **Then** the inspector, the time and the sample size are stored
- **Never** accepting an anonymous pass, which cannot be investigated when the complaints arrive

R09.3 An expiring certificate warns before it expires

- **When** a certificate approaches its expiry
- **Then** it is raised while there is still time to renew
- **Never** discovering the lapse at the moment a buyer asks for it

R09.4 A rejected batch cannot be sold as first quality

- **When** a batch is rejected
- **Then** it is marked and can only be sold through a channel that accepts seconds
- **Never** letting it re-enter the ordinary sellable pool

R09.5 A defect is attached to the design and the stage

- **When** a defect is recorded
- **Then** both the design and the stage that produced it are named
- **Never** recording it against the batch alone, which loses the pattern that would have prevented the next one

R09.6 A compliance document is evidence, not a checkbox

- **When** a compliance requirement is marked met
- **Then** the document proving it is attached
- **Never** accepting a tick with nothing behind it, which is what fails an audit

R09.7 A sustainability figure comes from the same evidence

- **When** an ESG figure is reported

- **Then** it is computed from the certificate and audit records already on file
- **Never** assembling it separately once a year from numbers nobody can trace

Module 10 · Warehouse — 8 rules

R10.1 A pick is confirmed against the bin it came from

- **When** an item is picked
- **Then** the bin is recorded on the movement
- **Never** decrementing a warehouse total with no bin, which makes the next cycle count unexplainable

R10.2 A short pick stops the pack, it does not silently reduce the order

- **When** the picker cannot find the full quantity
- **Then** the shortage is raised against the order and the pack waits
- **Never** packing what was found and invoicing for it as though that was the order

R10.3 A scan is the same event as a keyed entry

- **When** a code is captured by scanner, phone camera or typing
- **Then** the same movement is written
- **Never** having a scanning path that writes different records from the manual path

R10.4 A cycle count adjustment names a reason

- **When** a count differs from the system
- **Then** the adjustment records the reason and the person
- **Never** writing the system down to the counted figure with no explanation, which hides theft and damage equally well

R10.5 The packing video is linked to the shipment

- **When** a parcel is recorded on video at packing
- **Then** the recording is attached to that shipment
- **Never** keeping the footage in a folder by date, which makes it unusable in the dispute it exists for

R10.6 A bin holds a location, not a guess

- **When** stock is put away
- **Then** the destination bin is captured at put-away
- **Never** assigning a default bin so the step can be skipped

R10.7 A dispatch cut-off is per channel

- **When** a channel has a handover deadline
- **Then** the queue is ordered and warned against that channel's own cut-off
- **Never** applying one cut-off to all of them, which misses the earliest and idles for the latest

R10.8 A returned parcel is inspected before it is anything else

- **When** a return arrives at the warehouse
- **Then** it is booked into a return-inspection location first
- **Never** restocking on arrival, which puts an unchecked item back on sale

Module 11 · Logistics — 11 rules

R11.1 The courier rate is checked against the packed weight

- **When** a courier bills for a shipment
- **Then** the billed weight is compared with the packed weight recorded at packing
- **Never** accepting the courier's weight without comparison, which is the most consistently overcharged line in the business

R11.2 A weight dispute is raised with the evidence attached

- **When** billed and packed weight differ beyond tolerance
- **Then** a dispute is raised carrying the packing record
- **Never** absorbing the difference because each one is small

R11.3 An undelivered parcel is chased before it becomes a return

- **When** a delivery attempt fails
- **Then** the NDR is actioned within the window the courier allows
- **Never** letting it lapse into a return, which costs the freight twice and the sale once

R11.4 COD collected is a receivable until it is remitted

- **When** a COD parcel is delivered
- **Then** the amount is a receivable from the courier
- **Never** treating delivery as payment, which reports cash the business does not have

R11.5 A remittance is matched parcel by parcel

- **When** a courier remits COD
- **Then** each parcel in the remittance is matched individually
- **Never** accepting the total, which is how short remittances go unnoticed for months

R11.6 A manifest is a record, not a printout

- **When** parcels are handed over
- **Then** the handover is recorded against each shipment with the time and the person
- **Never** keeping only a signed sheet, which cannot be queried when a parcel is disputed

R11.7 An RTO parcel is stock again only after inspection

- **When** a return to origin is received
- **Then** it goes through inspection before it can be sold
- **Never** restocking it automatically on scan

R11.8 Freight cost reaches the order it belongs to

- **When** a shipment is costed
- **Then** the freight is attributed to the order
- **Never** holding freight only as a monthly expense, which makes per-order and per-channel profit fiction

R11.9 A courier can be changed without losing history

- **When** a courier is switched off
- **Then** every past shipment, AWB and dispute stays readable
- **Never** making history depend on an integration that is still connected

R11.10 A zone and rate card are dated

- **When** courier rates change
- **Then** the new card is stored with its effective date
- **Never** overwriting the card, which makes every past shipment look mischarged

R11.11 A partial-COD order has two collections and both are tracked

- **When** an order is placed with an advance online and the balance on delivery
- **Then** the advance is a receipt now and the balance is a receivable from the courier until it is remitted
- **Never** treating the advance as the whole payment, which makes every such order look settled while most of the money is still outstanding

Module 12 · Accounting & GST — 24 rules

R12.1 Money is an integer count of paise

- **When** any amount is held, added or compared
- **Then** it is an integer number of paise, becoming a decimal string only where a person reads it
- **Never** holding money in a floating-point number, where ₹0.10 + ₹0.20 is not ₹0.30 and a trial balance stops balancing

R12.2 An amount finer than a paisa is refused, not rounded

- **When** a computation produces a fraction of a paisa
- **Then** it is refused and the caller must round deliberately
- **Never** rounding silently, which is how two sides of the same figure drift apart and nobody can say which is right

R12.3 A split sums back to the original, exactly

- **When** an amount is divided — across lines, across companies, across periods
- **Then** the parts add back to the whole, with the round-off returned as its own figure
- **Never** losing or inventing a paisa in the split, and never hiding the remainder inside the largest part

R12.4 An unbalanced entry is refused, with the gap named

- **When** a voucher is posted whose debits and credits differ
- **Then** it is refused and the difference is stated
- **Never** posting it to a suspense account to make it balance, which converts an error into a permanent record

R12.5 A line cannot be a debit and a credit at once

- **When** a posting line carries both
- **Then** it is refused
- **Never** netting the two into whichever is larger

R12.6 The trial balance is computed, never stored

- **When** the trial balance is asked for
- **Then** it is summed from the posting lines at that moment
- **Never** reading a maintained total, which is a number that can be wrong without anything looking wrong

R12.7 A locked period refuses a backdated entry

- **When** a voucher is dated inside a closed period
- **Then** it is refused and the lock that stopped it is named
- **Never** posting it into the current period instead, which silently moves last year's result into this one

R12.8 Unlocking a period is itself recorded

- **When** a closed period is reopened
- **Then** who reopened it, when and why is written to the trail
- **Never** allowing a quiet reopen, which is the one action that could undo every other guarantee here

R12.9 A tax rate resolves on the date of the document

- **When** tax is computed for any invoice
- **Then** the rate in force on that document's date is used
- **Never** applying today's rate to an old invoice, which makes correct history look like an error

R12.10 Two rates covering one date is ambiguous, not a coin toss

- **When** two effective-dated rows overlap for the same date
- **Then** the resolution is refused and the overlap is named
- **Never** picking the newer one, which makes the answer depend on insertion order

R12.11 A voided entry is reversed, never erased

- **When** a posted voucher is wrong
- **Then** a reversing entry is posted and both stay visible
- **Never** editing or deleting the original, which is the difference between a correction and a cover-up

R12.12 Every figure clicks down to the record that produced it

- **When** any total appears on any screen

- **Then** it is a live query that can be opened down to its vouchers and their documents
- **Never** showing a figure that cannot be traced — an untraceable number is a defect, not a rounding difference

R12.13 An invoice number is sequential per company and per series

- **When** an invoice is raised
- **Then** it takes the next number in that company's series
- **Never** reusing, skipping or back-filling a number, which is the first thing a tax audit tests

R12.14 A GST return is built from vouchers, not from a summary

- **When** GSTR-1 or 3B is prepared
- **Then** it is computed from the underlying invoices
- **Never** accepting a typed summary figure, which cannot be reconciled when the portal disagrees

R12.15 ITC is claimed only where the supplier has filed

- **When** input credit is taken
- **Then** it is matched against the supplier's filed data and the unmatched part is held
- **Never** claiming everything and reversing later, which turns a reconciliation into a liability

R12.16 A place of supply decides the tax, not the billing address

- **When** GST is computed
- **Then** the place of supply determines CGST/SGST or IGST
- **Never** defaulting to the billing address, which mis-splits the tax on every drop-ship

R12.17 A credit note references the invoice it reverses

- **When** a credit note is raised
- **Then** the original invoice is named on it
- **Never** issuing a free-standing credit note, which cannot be matched in either set of books

R12.18 Depreciation is posted, not just calculated

- **When** a period closes
- **Then** depreciation is posted as an entry like any other
- **Never** showing it as a computed figure on a report while the ledger disagrees

R12.19 A company with no tax registration is still a company

- **When** a group company has no registration of its own
- **Then** it keeps its own books and joins the group figures
- **Never** dragging it into a return it does not belong in, and never leaving it out of the group result

R12.20 Year-end close locks, and the lock is the record

- **When** a financial year is closed
- **Then** the period is locked and the closing balances are carried forward as an entry

- **Never** leaving the year open indefinitely so late entries can drift in unnoticed

R12.21 Every voucher type posts through one engine

- **When** a sale, purchase, credit note, debit note, payment, receipt, journal, contra or counter sale is recorded
- **Then** all nine post through the same ledger routine
- **Never** giving a voucher type its own posting logic — this is where home-built accounting breaks and the modules stop agreeing about the same figure

R12.22 Net GST is input against output, per period, per company

- **When** the GST position for a period is computed
- **Then** it is output tax less eligible input credit for that company and that period
- **Never** netting across companies, which offsets one registration's liability with another's credit and is not a return anyone may file

R12.23 Money never becomes a float, in any layer

- **When** an amount is stored, moved between the engine and the database, or exported
- **Then** it stays an integer count of paise end to end, converted for display only
- **Never** a real, double, float or an unlabelled decimal column anywhere a money value lives

R12.24 A money column says what unit it is in

- **When** a column holds an amount
- **Then** its name ends in paise
- **Never** a column called total, amount or cost with no unit — the same name read as rupees by one developer and paise by the next is a factor of a hundred in the books

Module 13 · Treasury & Financial Planning — 8 rules

R13.1 A forecast never posts to the ledger

- **When** a cash-flow projection is produced
- **Then** it is held as a projection, separate from posted entries
- **Never** writing an expected receipt into the books, which reports money that has not arrived

R13.2 A bank line is matched to a voucher, not to a total

- **When** a bank statement is reconciled
- **Then** each line is matched to the entry that caused it
- **Never** reconciling on the closing balance alone, which hides two errors that happen to cancel

R13.3 An unmatched bank line stays visible until it is explained

- **When** a statement line cannot be matched
- **Then** it stays on the unreconciled list with its age
- **Never** writing it off to a sundry account to clear the screen

R13.4 A PDC is a commitment before it is cash

- **When** a post-dated cheque is received
- **Then** it is tracked as a commitment until it clears
- **Never** recognising it as cash on receipt

R13.5 Budget versus actual compares like with like

- **When** a variance is shown
- **Then** both sides use the same period, company and account basis
- **Never** comparing a full-year budget against a part-year actual without saying so

R13.6 A cash forecast names its assumptions

- **When** a projection is produced
- **Then** the collection and payment assumptions behind it are stored with it
- **Never** presenting a projection whose basis cannot be recovered a month later

R13.7 Inter-company funding is recorded on both sides

- **When** one group company funds another
- **Then** both companies post it, naming each other as counterparty
- **Never** recording it in one set of books only, which leaves the group permanently out of balance

R13.8 A currency amount keeps the rate it was converted at

- **When** a foreign-currency transaction is recorded
- **Then** the original amount, the currency and the rate used are all stored
- **Never** storing only the converted figure, which cannot be revalued or explained afterwards

Module 14 · Settlement — 13 rules

R14.1 A payout is matched line by line to orders

- **When** a marketplace settlement file arrives
- **Then** every line is matched to the order it belongs to
- **Never** accepting the net credited amount, which is how a short payment becomes invisible

R14.2 Every deduction is identified before the payout is accepted

- **When** commission, shipping, penalty, TCS or TDS is deducted
- **Then** each is posted to its own account
- **Never** posting the deductions as one lump, which makes an overcharge impossible to find

R14.3 A variance beyond tolerance raises a claim

- **When** the settled amount differs from the expected amount
- **Then** a claim is raised carrying the order, the expectation and the difference
- **Never** absorbing it because it is small — the small ones are the recurring ones

R14.4 A claim has a deadline and the deadline is tracked

- **When** a claim is raised
- **Then** the channel's filing window is stored and warned on
- **Never** letting a valid claim expire unfiled

R14.5 An expected settlement exists from the moment of the sale

- **When** an order is confirmed on a marketplace
- **Then** a settlement expectation is created then
- **Never** waiting for the payout to discover what should have arrived

R14.6 TCS and TDS are receivables, not costs

- **When** a marketplace deducts tax at source
- **Then** it is posted as a receivable against the tax authority
- **Never** expensing it, which understates profit and loses the credit

R14.7 A settlement is reconciled to the bank, not just to the file

- **When** a payout is recorded
- **Then** it is matched to the actual bank credit
- **Never** treating the settlement report as proof that the money arrived

R14.8 A re-sent settlement file does not double-post

- **When** the same settlement file is imported twice
- **Then** already-matched lines are recognised and skipped
- **Never** posting them again, which doubles both revenue and deductions

R14.9 A fee schedule is dated and compared against

- **When** a commission is deducted
- **Then** it is checked against the agreed rate in force on that date
- **Never** accepting whatever rate the file states, which is the single largest silent leak in marketplace trade

R14.10 A settled order is profitable or unprofitable at the SKU

- **When** a payout is fully matched
- **Then** the true net per SKU is computed after every deduction
- **Never** judging profitability on the listed price, which ignores the third of it that never arrives

R14.11 A claim that is paid closes against the original variance

- **When** a channel credits a claim
- **Then** it is matched back to the variance it settles
- **Never** posting the credit as unrelated income, which leaves the variance open forever

R14.12 A settlement figure never overwrites a sale figure

- **When** the settlement disagrees with the order
- **Then** both are kept and the difference is the variance
- **Never** adjusting the original sale to match the payout, which erases the evidence of the shortfall

R14.13 The realisation on a marketplace sale is the price minus every deduction

- **When** what a channel sale actually earned is computed
- **Then** it is the selling price less shipping, commission, fixed fee, GST on those fees, TCS and TDS — each taken from the settlement file
- **Never** judging a sale on its listed price, which ignores the part of it that never arrives, and never applying an assumed commission percentage when the file states the real one

Module 15 · E-commerce / OMS — 19 rules

R15.1 Companies and channels are read from the data, never from a list in the code

- **When** orders or sheets from any number of companies and channels are processed
- **Then** the companies and channels present are discovered and each gets its own columns
- **Never** writing a fixed set of companies or channels into the software, which caps the business at whatever it happened to have on the day the code was written

R15.2 A tenth or eleventh channel needs no code change

- **When** a new marketplace or company is added
- **Then** it is a row, and every figure, column and consolidation follows
- **Never** requiring a release to sell somewhere new

R15.3 A channel belongs to a company

- **When** two companies both sell on the same marketplace
- **Then** each has its own channel record, and both may use the same short code
- **Never** sharing one channel across companies, which merges two companies' sales into one figure

R15.4 A price is never invented for an item that has none

- **When** an item has no price on file
- **Then** it is reported as having no price and named in the summary
- **Never** substituting an average or a similar item's price, which quietly fabricates revenue

R15.5 Net is sale minus return, and inventory is net plus wrong return

- **When** quantities are rolled up
- **Then** net sale is sale minus return, and the inventory figure adds back the wrong returns
- **Never** treating a wrong return as ordinary saleable stock, because it is not the item that was sent out

R15.6 A blank cell is blank, not a value

- **When** a column contains only whitespace

- **Then** it is read as empty
- **Never** treating a lone space as a marked entry, which converts formatting into business fact

R15.7 An item that only ever came back is still reported

- **When** an item has returns but no sales in the period
- **Then** it appears with its returns
- **Never** dropping it because it has no sale line, which hides the worst-performing items entirely

R15.8 A totals row is the sum of the rows above it

- **When** a report shows a total
- **Then** it equals the rows it sits under
- **Never** computing the total by a different route from the detail, which is how a report disagrees with itself

R15.9 A marketplace order pull creates a real order

- **When** orders are fetched from a channel
- **Then** a sales order is created, stock is reserved, and the pick list follows
- **Never** holding channel orders in a staging area that has to be re-entered to become real

R15.10 A cancelled channel order releases its reservation

- **When** the channel cancels an order
- **Then** the reservation is released and the cancellation recorded
- **Never** leaving stock reserved against an order the channel has already dropped

R15.11 A wrong return is dead stock, not stock

- **When** a return is inspected and found to be a different or damaged item
- **Then** it is written to dead stock with its cost recognised as a loss
- **Never** restocking it as first quality, which sells a customer the same problem twice

R15.12 A listing rejected by a channel says why

- **When** a push to a channel fails
- **Then** the rejection and its reason are reported back against the listing
- **Never** reporting a push as successful when part of it failed, which leaves the business believing it is present where it is not

R15.13 A manual data check is a recorded step, not a habit

- **When** figures are checked by hand before a cycle closes
- **Then** the check, the person and the outcome are recorded
- **Never** relying on someone remembering to look

R15.14 A channel-specific SKU code never becomes the master code

- **When** a channel uses its own identifier

- **Then** it is stored as a mapping against our SKU
- **Never** adopting the channel's code as the item code, which breaks the moment a second channel does the same

R15.15 A size recommendation is advice, never a silent substitution

- **When** a fit suggestion is offered
- **Then** it is shown as a recommendation the customer chooses
- **Never** changing the size on an order on the customer's behalf

R15.16 An order held past its cut-off is escalated, not queued

- **When** an order approaches the channel's dispatch deadline
- **Then** it is raised to the person who can act, naming the deadline
- **Never** letting it age quietly into a penalty

R15.17 Closing stock is opening plus in minus out

- **When** a stock position is computed for a period
- **Then** closing = opening + receipts – issues, from the movements themselves
- **Never** carrying a maintained closing figure that can drift from the movements that produced it

R15.18 Courier return, customer return and wrong return cost three different things

- **When** a return is processed
- **Then** a courier return costs repacking only, a customer return costs alteration plus iron plus packing at the rate set for that design, and a wrong return is written off at the full selling price
- **Never** applying one blended return cost to all three, which hides the expensive kind inside the cheap kind

R15.19 A wrong return is never added back to stock

- **When** a return is found to be a different item from the one sent
- **Then** it becomes dead stock and the selling price is recognised as a loss
- **Never** restocking it, at any value, however sellable it looks

Module 16 · HR & Payroll — 22 rules

R16.1 A raise closes the old row, it does not overwrite it

- **When** a salary or rate changes
- **Then** the row in force is closed on the day before, and a new row opens
- **Never** editing the existing figure, which rewrites what the person was actually paid last year

R16.2 History resolves to what was actually in force

- **When** a past month is recomputed
- **Then** the rate in force in that month is used
- **Never** recomputing an old payslip at today's rate

R16.3 A future-dated raise activates by itself

- **When** a raise is entered with a future date
- **Then** it takes effect when that month arrives, with nobody remembering to apply it
- **Never** requiring a manual step, which is how an agreed raise is missed

R16.4 A month with nothing in force raises, and never returns zero

- **When** no rate covers the month being computed
- **Then** the computation is refused and the gap is named
- **Never** returning zero, which pays a real person nothing and looks like a valid answer

R16.5 Backdating over an open row is refused

- **When** a change is entered with a date inside a period already settled
- **Then** it is refused
- **Never** silently rewriting history that has already been paid and posted

R16.6 A person can leave and come back

- **When** someone rejoins after a break
- **Then** the spell log holds both periods and the gap between them
- **Never** creating a second employee record, which splits their history and their service

R16.7 Month spans handle February and the year end

- **When** a period is computed across month or year boundaries
- **Then** the real calendar is used
- **Never** assuming thirty-day months, which is wrong twelve times a year and badly wrong in February

R16.8 Staff and piece-rate workers sit in one register

- **When** payroll is prepared
- **Then** monthly staff and piece-rate karigars are computed in the same run and paid from the same register
- **Never** running two payrolls that have to be added together by hand

R16.9 An advance is recovered against a named advance

- **When** a deduction is made at payout
- **Then** it names the advance it is recovering and reduces that balance
- **Never** deducting an amount that cannot be traced to a specific advance

R16.10 Attendance drives pay, and both are visible together

- **When** a payout is computed
- **Then** the attendance it was computed from is shown beside it
- **Never** presenting a pay figure whose basis the person being paid cannot see

R16.11 Identity documents are read, never stored in a file that leaves

- **When** Aadhaar, PAN, bank or UPI detail is used for a computation
- **Then** it is used and not serialised into any exported or committed artifact
- **Never** writing personal identifiers into a report, a backup file or a repository

R16.12 A payout that fails to post does not mark as paid

- **When** the bank transfer or the ledger posting fails
- **Then** the payout stays unpaid and the failure is raised
- **Never** marking it paid on submission, which loses a real person's wages in the gap

R16.13 The daily rate is the monthly salary divided by twenty-seven

- **When** a day of attendance is priced
- **Then** the daily rate is that month's salary $\div$ 27, using the salary in force in that month
- **Never** using calendar days, working days, or a rate carried over from a month with a different salary

R16.14 Attendance codes have fixed multipliers and a blank is absent

- **When** earned pay is computed from attendance
- **Then** present, holiday, on-duty and paid leave count 1, a half day counts 0.5, absent and unpaid leave count 0, and an empty cell counts as absent
- **Never** treating a blank as present, or as unknown to be filled in later — a blank that pays is a blank that will be left blank

R16.15 Threshold hours do not move when salary moves

- **When** a raise takes effect
- **Then** the monthly hour threshold for that role stays as it was
- **Never** scaling the threshold with the salary, which silently changes what the person is expected to work in exchange for a raise

R16.16 Productivity cost is that month's salary over the threshold, times hours worked

- **When** the cost of a person's time is charged to work
- **Then** it is (salary in force that month $\div$ threshold hours) $\times$ the hours actually active
- **Never** using a single annual figure, which misprices every month on either side of a raise

R16.17 A holiday is paid and produces no hours

- **When** a holiday is marked
- **Then** it pays a full day and contributes zero productive hours
- **Never** counting holiday hours as production, which flatters every efficiency figure that reads them

R16.18 A half day is half the hours, from the same start

- **When** a half day is marked
- **Then** it starts at the normal in-time and its hours are half the full shift for that person's pattern
- **Never** assuming a fixed midday finish for everyone, when the male and female shift lengths differ

R16.19 The festival flag drives leave and nothing else

- **When** a religion is recorded against a person
- **Then** it is used only to match a festival-leave request
- **Never** using it as a filter, a grouping or a report dimension anywhere else in the system

R16.20 A geofence failure flags, it does not refuse

- **When** attendance is marked outside the radius set for the unit, or outside the grace window
- **Then** it is recorded with the flag and raised to the manager
- **Never** refusing the mark — a system that locks someone out of being paid for standing at the wrong gate has failed at its actual job

R16.22 A shared document carries the pay rules, never the pay roster

- **When** a plan, a specification or any document that leaves this building is generated
- **Then** it carries the formulas, thresholds and effective-dating that decide pay, and refers to the roster rather than reproducing it
- **Never** printing an individual's name beside their salary, or their religion at all, into a document that is committed to a repository and travels with every copy — the software needs those fields, a reader of the plan does not

R16.21 An override is allowed and is always recorded

- **When** an administrator corrects attendance, a geofence flag or a payroll figure
- **Then** the change, the person and the reason go to the audit trail
- **Never** an override that leaves no trace, which is indistinguishable from the system having been wrong

Module 17 · Marketing — 10 rules

R17.1 A campaign is measured on revenue, not on opens

- **When** campaign performance is reported
- **Then** it is attributed to actual orders
- **Never** reporting opens and clicks as the result, which measures the message rather than the business

R17.2 A repricing rule shows what it did

- **When** a rule changes a price
- **Then** the change, the rule that made it and the effect on orders are recorded together
- **Never** changing prices with no record, which makes a bad rule impossible to identify or reverse

R17.3 A price floor is a floor

- **When** a repricing rule would go below the floor set for a SKU
- **Then** it stops at the floor
- **Never** undercutting to match a competitor below cost

R17.4 A markdown starts before the stock is dead, not after

- **When** stock reaches the age set for it
- **Then** the markdown schedule begins
- **Never** waiting until it is unsellable, which converts a lower-margin sale into a write-off

R17.5 A campaign cannot message someone who has not consented

- **When** a marketing send is prepared
- **Then** the recipient list is filtered by consent at send time
- **Never** sending to a list captured before the consent was checked

R17.6 A published page reads live catalogue data

- **When** a page shows a price or a stock state
- **Then** it reads the same record the order screen reads
- **Never** pasting a figure into the page, which goes stale the first time the price changes

R17.7 An exhibition is a channel

- **When** leads and sales come from a trade show
- **Then** they land in CRM and the order book against that channel
- **Never** collecting them on paper to be entered later, which is where they are lost

R17.8 A marketing automation cannot move money

- **When** a campaign rule fires
- **Then** it may message, tag, schedule or reprice within its limits
- **Never** issuing a refund, a credit note or a payment — that is not what this engine is allowed to do

R17.9 A scheduled post that fails is reported as failed

- **When** a scheduled publication does not go out
- **Then** it is raised with the reason
- **Never** showing it as published in the calendar while nothing was posted

R17.10 Return on ad spend is measured against real orders

- **When** campaign performance is computed
- **Then** it is revenue from attributed orders ÷ spend actually incurred
- **Never** using a platform's own reported conversions as the revenue figure, which counts orders this system has no record of

Module 18 · AI Content Engine — 11 rules

R18.1 Content is written from the catalogue, not about the category

- **When** a listing or description is generated
- **Then** it is generated from that product's own attributes

- **Never** writing plausible copy about the kind of thing it is, which is how a listing describes features the product does not have

R18.2 Structured fields get keywords; anything a human reads gets feeling

- **When** text is produced for a back-end field versus a caption
- **Then** each is written for its own reader
- **Never** writing both the same way, which is the clearest signal of machine-written content

R18.3 Product nouns are banned from creative surfaces

- **When** a caption or a hook is written
- **Then** the product noun is excluded
- **Never** letting search vocabulary bleed into copy meant to be felt

R18.4 The engine criticises its own draft before anyone sees it

- **When** a draft is produced
- **Then** it is put through the self-critique pass and the second draft is what is shown
- **Never** showing the first attempt, which is rarely the best one

R18.5 A render is seeked, never recorded

- **When** a video is produced from a page
- **Then** the clock is driven by hand and each frame is captured at its exact instant
- **Never** playing the animation and recording the screen, which bakes whatever else the machine was doing into the customer's reel

R18.6 The same scene renders to the same file

- **When** a render is repeated
- **Then** the output is identical to the byte
- **Never** producing a different file each time, which makes the output impossible to check or approve

R18.7 A generated asset is labelled as generated

- **When** an image or video is produced by a model
- **Then** it carries that fact in the asset record
- **Never** letting a generated image become indistinguishable from a photograph of the actual product

R18.8 Generation stays badged a mockup until a real provider is wired

- **When** a capability is demonstrated without a live provider behind it
- **Then** it is labelled a mockup wherever it appears
- **Never** showing a simulated render as a finished one

R18.9 Image generation states that it needs a graphics card

- **When** the image generation slot is opened with no provider attached

- **Then** it says so plainly and produces nothing
- **Never** presenting a finished-looking screen that cannot generate anything

R18.10 A cloned voice needs the consent of the person it came from

- **When** a voice is cloned for narration
- **Then** that person's recorded consent is on file against them
- **Never** cloning from a recording merely because it was available

R18.11 A publish reports what actually went live

- **When** content is pushed to several destinations
- **Then** each result comes back individually, with reasons for rejections
- **Never** reporting one overall success, which leaves the business absent where it believes it is present

Module 19 · SEO, AEO & AIO — 6 rules

R19.1 Structured data describes what is actually on the page

- **When** schema markup is generated
- **Then** it is generated from the same record the page renders
- **Never** marking up a price or availability that differs from the page, which is penalised and deserved

R19.2 A ranking figure names where it was measured

- **When** a position or citation is reported
- **Then** the engine, the query and the date are stored with it
- **Never** reporting a bare position, which cannot be compared with anything

R19.3 An answer-shaped page still says the same thing as the product record

- **When** content is shaped to be quoted by an answer box
- **Then** the claims match the catalogue
- **Never** writing a more quotable claim than the product supports

R19.4 A technical fix is verified on the live page

- **When** a technical SEO issue is marked resolved
- **Then** the live page is re-fetched and re-checked
- **Never** closing it because the change was deployed

R19.5 AI-engine visibility is tracked over time, not sampled once

- **When** citation in an AI answer is measured
- **Then** it is measured repeatedly and stored as a series
- **Never** quoting a single lucky result as the position

R19.6 A sitemap lists only pages that exist and are meant to be found

- **When** a sitemap is generated
- **Then** it contains live, indexable pages
- **Never** listing archived or blocked pages, which wastes the crawl on nothing

Module 20 · Projects & Collaboration — 9 rules

R20.1 Billable time becomes an invoice line without retyping

- **When** approved time exists against a project
- **Then** the rate card turns it into an invoice line and a real cost
- **Never** re-entering hours into an invoice, which is where the two figures start to differ

R20.2 Billable and non-billable are separated at entry

- **When** time is recorded
- **Then** it is marked billable or not as it is entered
- **Never** deciding at invoice time, which quietly turns unbillable work into a charge

R20.3 An approval shows the rule that demanded it

- **When** anything lands in the approvals queue
- **Then** the rule that sent it there is displayed beside it
- **Never** presenting a request with no stated reason, which makes approval a formality

R20.4 An approval decision goes to the audit trail

- **When** a request is approved or refused
- **Then** the decision, the person and the time are recorded
- **Never** recording only the outcome on the record, which loses who accepted the risk

R20.5 An automation run is kept step by step

- **When** a rule fires
- **Then** what triggered it, each step, and what each step returned are stored
- **Never** keeping only the outcome — an automation nobody can inspect afterwards is a rule the business cannot trust with its money

R20.6 An automation acts within a named scope

- **When** a rule is built
- **Then** the records it may read and write are declared on it
- **Never** letting a rule reach anywhere in the system because it happens to run as an administrator

R20.7 A project cost includes the time and the material

- **When** project profitability is computed
- **Then** labour, material and expenses booked to it are all included
- **Never** reporting on revenue and time alone, which shows a loss-making project as profitable

R20.8 A decision is recorded where the decision was made

- **When** a discussion resolves something
- **Then** it is attached to the record it concerns
- **Never** leaving the reasoning in a chat thread that will not be found in a year

R20.9 A procedure is scoped to the role it applies to

- **When** a standard procedure is published
- **Then** it is scoped to the role that performs it
- **Never** publishing one undifferentiated manual that nobody reads

Module 21 · Dashboard & BI — 9 rules

R21.1 The group figure is the sum minus inter-company trade

- **When** several companies are consolidated
- **Then** entries naming a counterparty inside the group are eliminated, and gross, eliminated and group are all shown
- **Never** presenting the plain sum as the group result, which inflates turnover by trade the group never did with the outside world

R21.2 An entry cannot be its own counterparty

- **When** an entry names a counterparty company
- **Then** it is refused if that is the same company
- **Never** allowing a company to trade with itself, which eliminates a figure that was never doubled

R21.3 The number of companies is data, not a constant

- **When** the group grows
- **Then** a company is a row and every consolidation follows
- **Never** building around a fixed number of companies or channels

R21.4 Every dashboard figure is a live query

- **When** a KPI is displayed
- **Then** it is computed from the ledger and the stock table at that moment
- **Never** reading a maintained summary table, which can be wrong without looking wrong

R21.5 A consolidated row is a formula over the company rows

- **When** a workbook or report shows a consolidated figure
- **Then** it is computed from the company rows beside it
- **Never** typing a separate consolidated total, which is a second copy that will disagree

R21.6 A figure a user may not see is not returned

- **When** a report runs for a scoped user

- **Then** out-of-scope rows are excluded from the query
- **Never** computing the full figure and hiding part of it in the display

R21.7 An exported report says when it was taken

- **When** a report is exported
- **Then** the as-at time and the filters are printed on it
- **Never** producing an undated export, which is quoted months later as though it were current

R21.8 A saved report keeps its definition, not its results

- **When** a report is saved and re-run
- **Then** the definition re-runs against current data
- **Never** storing a snapshot and presenting it as live

R21.9 A figure with no drill-down is a defect

- **When** any total is shown
- **Then** it opens to the records beneath it
- **Never** shipping a number that cannot be explained by clicking it

Module 22 · AI Assistant, Agents & Automation — 15 rules

R22.1 An answer carries the records it came from

- **When** the assistant answers a question about a figure
- **Then** the rows it used are attached and each one opens to its record
- **Never** giving a bare number, which cannot be checked and therefore cannot be trusted

R22.2 An unknown answer is said, never estimated

- **When** the assistant cannot find the figure
- **Then** it says so and shows what it looked at
- **Never** producing a plausible number — a confident wrong figure costs far more than an honest blank

R22.3 The assistant answers only from what the asker may already see

- **When** a question is asked by a scoped user
- **Then** retrieval is filtered to that user's permissions before the answer is composed
- **Never** letting the assistant become a way around permissions that every other screen enforces

R22.4 An agent cannot widen its own scope

- **When** an agent runs
- **Then** it works within the records and the spend it was given
- **Never** expanding its scope mid-run, however sensible the next step would be

R22.5 Money never moves without a human yes

- **When** an agent proposes a refund, a payment, a payout or a credit note
- **Then** it stops and waits for a person
- **Never** executing it, no matter how confident or how small the amount

R22.6 A customer is never messaged by an agent without approval

- **When** an agent drafts a message to a real customer
- **Then** a person approves it before it is sent
- **Never** sending on the agent's own judgement

R22.7 A price is never changed by an agent alone

- **When** an agent proposes a price change
- **Then** it enters the approvals queue with the reasoning attached
- **Never** writing the new price directly

R22.8 Every agent run is replayable step by step

- **When** an agent finishes, stops or fails
- **Then** what started it, what it read, what it proposed and what was approved are all recorded
- **Never** keeping only the outcome — an unexplained change made by software is worse than one made by a person

R22.9 Agent spending goes through the same ceiling as everything else

- **When** an agent calls a paid provider
- **Then** it is routed through the Provider Router and refused past the ceiling
- **Never** giving an agent its own unmetered budget

R22.10 The chatbot hands over rather than guessing about money

- **When** a customer asks about a refund, a charge or a complaint
- **Then** it hands to a person with the whole conversation attached
- **Never** answering from a general idea of the policy

R22.11 The chatbot never asks a customer for a credential

- **When** a customer is identified in a chat
- **Then** identity is established through the order and the contact already on file
- **Never** asking for a card number, a bank detail or a password — the promise made everywhere else does not get a chatbot-shaped exception

R22.12 A handover lands in the existing queue

- **When** a conversation is passed to a person
- **Then** it enters the Module 04 Helpdesk queue with its history
- **Never** creating a second inbox that someone has to remember to watch

R22.13 An agent is not a hidden actor in the audit trail

- **When** an agent changes anything
- **Then** the change is attributed to the agent, its run, and the person who approved it
- **Never** recording it under a service account, which makes an automated change indistinguishable from a human one

R22.14 A retrieved document does not become an instruction

- **When** the assistant reads a document, a review or a message while answering
- **Then** that content is treated as data to report on
- **Never** following instructions found inside retrieved content, which is how a supplier's PDF ends up steering the system

R22.15 An assistant answer is reproducible from the records it cites

- **When** the assistant states a figure
- **Then** re-running the same query over the same records gives the same figure
- **Never** an answer that cannot be reproduced, which is a guess with citations attached

10.1 · Decide which discretionary rules you want on WITH YOUR TEAM

Some rules are yours to choose — whether a price below a floor needs approval, whether a credit sale reserves the limit at once. Others can never be switched off by anybody, because somebody else relies on them.

Changing it: Turn a discretionary rule on or off any time, from a date. Transactions already posted are not re-judged against a rule that did not apply to them.

Done when: Every discretionary rule has been considered and set deliberately, rather than left at whatever it defaulted to.

Part 11 · What you are not locked into

You do not choose any of the parts below — that is the platform's job. You are entitled to know what they are anyway, because every one of them names what could take its place.

This is the promise: changing a supplier underneath you does not change your work. Not your screens, not your records, not your reports, not a rupee of anything already posted. *Neeche ka saamaan badla toh bhi aapka kaam waisa ka waisa.*

backend — The part of the software you never see, which does the actual work — checks the rules, saves the records, calculates the totals. *Hotel ka kitchen. Customer nahin dekhta, par khaana wahin banta hai.*

frontend — The part you see and click — the screens, the buttons, the forms. *Hotel ka dining hall aur menu card. Jo aapke saamne hai.*

fallback — The next option the system automatically moves to when the first one fails or is unavailable. *Bijli gayi toh inverter. Inverter gaya toh mombatti. Andhera kabhi nahin hota.*

interface — A written promise about what a part of the system does, without saying which product does it — so the product underneath can be swapped without anything above noticing. *Bijli ka socket. Socket ka*

size fix hai; usme koi bhi company ka plug lag jaata hai. **adapter** — A small piece of code that translates between the system and one outside service, so the rest of the system never has to know which service is in use. Travel adapter. Andar ka appliance wahi, bas plug ko us desk ke socket ke hisaab se badal diya. **cache** — A small, fast copy of information that was just looked up, kept ready in case it is asked for again. Counter pe rakha hua sabse zyada bikne wala saamaan. Har baar godown tak jaana nahin padta. **environment** — A separate running copy of the system — one for trying things, one that customers actually use. Rehearsal aur asli show. Practice alag jagah, taaki galti sabke saamne na ho. **uptime** — How much of the time the system is actually working and reachable. Dukaan mahine mein kitne din khuli rahi. Band rahi toh customer wapas chala gaya.

The database — PostgreSQL

What it does. Keeps every record — customers, orders, stock, vouchers — and answers questions about them.

Why this one. PostgreSQL is open source, runs anywhere, and has the two things this design needs built in: locks at the record level so one business cannot read another's rows, and exact whole-number arithmetic so money never drifts. Any managed Postgres service is a hosting decision, not a database decision — the same schema runs on all of them.

What can replace it

- A managed Postgres service — same database, somebody else runs the machine
- Postgres on your own server — the software is free, you supply the machine
- MySQL or MariaDB, if a team already knows them well — the schema needs rework for the record-level locks

The rest of the code only ever talks to `DatabaseService` — so changing the line above changes one file, not the application.

What the move actually costs. Moving between Postgres hosts is a dump and a restore. Moving off Postgres entirely means rewriting the isolation layer, which is the one part worth not moving.

File storage — Any S3-compatible object store

What it does. Keeps photographs, invoices and scanned documents — the things too big to sit in the database.

Why this one. Almost every file service speaks the same request format, so one adapter reaches most of them. That makes this the cheapest layer in the whole system to change your mind about.

What can replace it

- A different S3-compatible provider — usually a URL and a key change
- Files on your own server's disk, with a backup copy elsewhere
- A self-hosted object store such as MinIO, which speaks the same format

The rest of the code only ever talks to `FileStore` — so changing the line above changes one file, not the application.

What the move actually costs. Copy the files across and change the address. Nothing above this layer notices.

Cache and short-term memory — Redis, or a Redis-compatible store

What it does. Holds recently used answers and sign-in sessions so common screens open instantly.

Why this one. Nothing here is the only copy of anything. If the cache is wiped the system simply asks the database again and is a little slower for a minute — so this layer can be replaced, restarted or removed entirely without risking a single record.

What can replace it

- Valkey — the open-source continuation of the same thing, same commands
- Memory inside the application itself, which is enough until traffic grows
- A database table, slower but with nothing extra to run

The rest of the code only ever talks to `CacheService` — so changing the line above changes one file, not the application.

What the move actually costs. Near zero by design. Losing the cache loses no data, which is the whole reason it is safe to change.

The backend runtime — Node.js with TypeScript

What it does. Runs the business rules, checks permissions, writes records and calculates totals.

Why this one. The same language runs on the browser side, so one team can work across the whole system and code that validates a form can be shared with the code that validates the saved record — no rule gets written twice and no two versions of it drift apart.

What can replace it

- Any container host — the code is ordinary and carries no host-specific parts
- Python or Go for a service that genuinely suits them, talking over the same API
- A different Node framework — the business logic sits outside the framework on purpose

The rest of the code only ever talks to `the HTTP API contract` — so changing the line above changes one file, not the application.

What the move actually costs. Low, because the rules live in plain functions rather than inside a framework. Moving a service means moving the functions and putting a different door in front of them.

The API — REST over HTTPS, with a written schema

What it does. The agreed way the screens, the mobile view and any outside system ask the backend for things.

Why this one. Ordinary web requests over predictable addresses. Anything can call it — a browser, a phone, a spreadsheet, another company's software — without a special library.

What can replace it

- GraphQL for read-heavy screens, over the same underlying services
- A direct connection for live screens that must update by themselves
- Scheduled file exchange for partners who cannot call an API at all

The rest of the code only ever talks to `the published API schema` — so changing the line above changes one file, not the application.

What the move actually costs. Adding a second style is additive — the services underneath do not change.

The frontend — React with TypeScript, screens generated from configuration

What it does. Everything a person sees and clicks — screens, forms, tables, dashboards.

Why this one. Screens are drawn FROM SETTINGS rather than written one by one. A tenant that renames a field, adds a column or turns a module off gets a different screen with no new code written — which is the only way one system can serve a steel plant and a single creator without becoming two systems.

What can replace it

- Vue or Svelte — the screen definitions are plain data and do not care what draws them
- Server-rendered pages where speed on a weak connection matters more than interaction
- A native mobile shell reading the same screen definitions

The rest of the code only ever talks to `the screen definition format` — so changing the line above changes one file, not the application.

What the move actually costs. Moderate, and bounded: what a screen contains is data, so a rewrite replaces the painter, not the paintings.

Background work — A queue backed by the database, with named workers

What it does. Does the things nobody should have to wait for — sending a thousand messages, building a month-end report, pulling orders overnight.

Why this one. Every job is written so that running it twice does the same thing as running it once. That single discipline is what makes it safe to retry after a failure, and it is worth more than any particular queue product.

What can replace it

- A Redis-backed queue when volume outgrows the database
- A hosted queue service, behind the same interface
- An external workflow tool such as n8n for steps a non-programmer should be able to edit

The rest of the code only ever talks to `JobQueue` — so changing the line above changes one file, not the application.

What the move actually costs. Low. Jobs are plain functions with a name; the queue only decides when they run.

Search — PostgreSQL full-text search

What it does. Finds a product, a customer or a document by a few typed letters, instantly.

Why this one. Postgres can search well enough for a long time, and starting there means one less thing running, one less thing to back up, and one less thing to keep in step with the database.

What can replace it

- OpenSearch or Elasticsearch when catalogues grow large
- Meilisearch or Typesense — small, fast, self-hostable
- A hosted search service behind the same interface

The rest of the code only ever talks to `SearchService` — so changing the line above changes one file, not the application.

What the move actually costs. Low, and it is a one-way door you can walk back through: the records stay in the database either way, so a search engine is only ever a faster copy.

Sign-in and permissions — Sessions issued by the platform, with permissions checked in the backend and again in the database

What it does. Proves who somebody is, then decides what they are allowed to see and change.

Why this one. Who you are and what you may do are kept apart deliberately. Sign-in can be handed to an outside service — or to a customer's own company login — while permissions stay ours, because they depend on the company and role structure no outside service knows about.

What can replace it

- An identity provider for sign-in only, with permissions still decided here
- A customer's own company sign-in, for enterprises that require it
- Self-hosted Keycloak or Authentik, when nothing may leave the building

The rest of the code only ever talks to `IdentityService` — so changing the line above changes one file, not the application.

What the move actually costs. Low for sign-in, by design. Permissions never move, so the expensive half is never in play.

Keys and passwords the system uses — Environment variables on the server, readable only by the service account

What it does. Holds the connection details and keys the software needs, away from the code.

Why this one. A key in the code is a key in every copy of the code forever. Keeping them outside means one can be replaced in a minute without changing a line.

What can replace it

- A managed secrets service, when there are enough of them to be worth it
- Self-hosted Vault or Infisical
- Encrypted files kept outside source control

The rest of the code only ever talks to `ConfigService` — so changing the line above changes one file, not the application.

What the move actually costs. Very low — the code asks for a name and does not care where the value came from.

Messages to customers and staff — A message service with one adapter per provider, per tenant

What it does. Sends WhatsApp messages, text messages and email — reminders, confirmations, statements.

Why this one. Each tenant connects its own accounts. The platform is built with a place for them to plug in and never holds one central account of its own — a business's conversations with its own customers belong to that business. The platform's job is the plug, not the account.

What can replace it

- Any WhatsApp provider — the adapter changes, the code that decides what to send does not
- Text message and email as fallbacks when a message cannot be delivered
- A shared inbox or an export, for a tenant with no messaging account at all

The rest of the code only ever talks to `MessageService` — so changing the line above changes one file, not the application.

What the move actually costs. One adapter per provider. Switching is a settings change made by the tenant, not a release made by us.

Storefronts and marketplaces — A channel adapter per storefront or marketplace

What it does. Brings orders in from a shop website or a marketplace, and sends stock and prices back out.

Why this one. Every one of these is treated as a channel with an adapter. Adding a marketplace is writing one adapter and creating one record — never a change to how orders work.

What can replace it

- A different storefront platform — a new adapter, and orders keep arriving
- File import for a channel with no connection available
- Manual entry, which must always remain possible

The rest of the code only ever talks to `ChannelAdapter` — so changing the line above changes one file, not the application.

What the move actually costs. One adapter each. The order, the stock number and the books never change shape.

Taking payments — A payment adapter per provider, with the card field hosted by the provider

What it does. Collects money from customers online.

Why this one. Card details are handed straight to the payment provider's own secured field and never touch this system — so there is nothing sensitive here to protect, and switching provider moves no card data, because none was ever held.

What can replace it

- Any other payment provider, behind the same interface
- Bank transfer and UPI details recorded against the invoice
- Cash on delivery, reconciled when the courier settles

The rest of the code only ever talks to `PaymentService` — so changing the line above changes one file, not the application.

What the move actually costs. One adapter. No card data ever moves, because none is ever stored.

Delivery and couriers — A courier adapter per carrier

What it does. Books a shipment, prints the label, and follows it to the door.

Why this one. Rate cards and tracking differ per courier; what a shipment IS does not.

What can replace it

- A courier aggregator, which is itself just one more adapter
- A different carrier directly
- Manual booking with the tracking number typed in — always available

The rest of the code only ever talks to `CourierService` — so changing the line above changes one file, not the application.

What the move actually costs. One adapter each.

Artificial intelligence — A router in front of several providers, ending on one that needs nothing bought

What it does. Writes descriptions, tags photographs, summarises, and answers questions about your own data.

Why this one. Ordered fallback, a breaker on anything failing repeatedly, and a spend ceiling that REFUSES rather than warns. Because every capability also has an option that costs nothing, a spent budget can stop the spending without ever stopping the business.

What can replace it

- Any hosted model provider — an entry in the router, not a change to the system
- A model running on your own machine, for work that is routine or private
- Templates and rules with no model at all, which must always remain the last resort

The rest of the code only ever talks to `ModelRouter` — so changing the line above changes one file, not the application.

What the move actually costs. A list entry. The router exists precisely so changing provider is never a project.

Where it runs — Containers on a virtual server

What it does. The machines that serve the website and the application.

Why this one. The application is packaged as an ordinary container with nothing host-specific inside it. That single decision is what keeps every hosting option open, forever.

What can replace it

- A managed container platform, when scaling by hand stops being fun
- A different cloud, or a different country, for the same container
- A machine in your own office, for data that must not leave it

The rest of the code only ever talks to `the container image` — so changing the line above changes one file, not the application.

What the move actually costs. Low by construction. If moving hosts is ever hard, something host-specific has leaked in and that is the bug.

Source control and automatic checks — Git, with automatic checks on every change

What it does. Keeps the history of every change and runs every test before anything goes live.

Why this one. Git itself is the thing that matters, and git is not owned by anybody. The host is a convenience.

What can replace it

- A different hosting service — a git repository moves with one command
- Self-hosted Gitea or Forgejo
- A separate build service reading the same repository

The rest of the code only ever talks to `the test commands themselves` — so changing the line above changes one file, not the application.

What the move actually costs. Very low. The checks are ordinary commands, so any system that can run a command can run them.

Watching it — Structured logs and error reporting, in an open format

What it does. Reports errors, measures speed, and tells you when something stops answering.

Why this one. Standard formats mean the tool that reads them is replaceable without changing what the system emits.

What can replace it

- Any hosted error-tracking service
- Self-hosted GlitchTip, or a Grafana and Prometheus stack
- Log files plus an uptime checker, which is enough at the start

The rest of the code only ever talks to `Logger and the metric format` — so changing the line above changes one file, not the application.

What the move actually costs. Low — the system emits a standard shape and does not know who is reading it.

Making documents — HTML templates printed to PDF by a headless browser

What it does. Produces invoices, statements, labels and reports as files a person can print or send.

Why this one. What a document SAYS is data. How it is drawn is replaceable, and should be.

What can replace it

- A dedicated PDF library for very high volume
- A hosted document service
- Spreadsheet or CSV output, which some readers prefer anyway

The rest of the code only ever talks to `DocumentRenderer` — so changing the line above changes one file, not the application.

What the move actually costs. Low. Templates are content; the renderer is a tool.

11.1 · Read the alternatives column, once, before you commit **WITH YOUR TEAM**

The question worth asking of any system you are about to run your business on is not "what is it built with" — it is "what happens to me if that thing goes away, doubles its price or changes its terms". Every layer answers that in writing, and each names at least two replacements rather than one.

Done when: Nothing your business depends on rests on a single supplier with no named replacement.

11.2 · Ask what a swap would actually cost you **WITH YOUR TEAM**

The honest answer is not always "nothing", and every layer says so in its own words — some moves are a copy across, one or two are genuinely expensive. What is constant is that the swap is a decision made on the platform side, and your records, your screens and your closed months are untouched by it either way.

Done when: You know which layer moves are cheap and which are not, and that none of them reach your data.

11.3 · Take your data out whenever you want, without asking anybody **IN THE APP**

The strongest form of not being locked in is being able to leave. Every list, every report and every ledger exports in an ordinary format that opens in a spreadsheet, and the export is a button you press yourself rather than a request somebody has to approve.

Changing it: Export any list at any time. Nothing is held back, and no export needs permission from the platform.

Done when: Every screen carrying a list can put that list in a file you own, in a format that opens without this software.

Part 12 · Everything you get, named

The whole list, not a count. A count tells you how much you are not being shown.

A **module** is a part of the business; an **app** is one screen and the work behind it. Every one of them can be switched off for a business that does not need it, and switched back on later with nothing lost — see Part 9.

industry pack — A settings file that teaches the system your trade — what you call things, the stages your work moves through, the documents you issue. *Ek hi machine, alag-alag saancha. Saancha badal do, wahi machine doosri cheez banane lagti hai.*

Module 01 · Platform — 8 apps

- **Identity, Settings & Audit** — Users, roles and permissions, company switching, tax and numbering setup — and an immutable record of everything that ever happened.
- **Industry Packs** — What your trade calls things, the stages your work moves through, the extra fields your records need, the documents you issue and the reference data you start with — all of it loaded as a row of configuration, never as a separate version of the software. Pick the pack that matches your trade and the whole system changes its words: the same order screen reads as a job, a matter, a consignment or an appointment, with the same columns underneath. A pack may rename what exists, add fields to tables that exist, and switch discretionary rules off; it may never invent a concept, add a field to a table that does not exist, contain any executable code, or switch off the guarantees — company scoping, the audit trail, money never being a float — because a trade may change its vocabulary and may not opt out of the things the books are trusted for. Adding a trade nobody anticipated is therefore a file somebody writes, not a release somebody ships.
- **Ask & Print** — Ask from your phone, anywhere: a ledger, a bill, today's packing slips. It comes back as a PDF — or prints on the office printer, with nothing plugged into your phone.
- **Communications** — WhatsApp commands and broadcasts, email and SMS, and the handful of scheduled jobs that carry a nudge to the right person without anyone remembering to send it. Every capability here has more than one interchangeable provider — none of them is ever the source of a figure the business reports.
- **WhatsApp Command Console** — The shop floor does not open a laptop. A worker or a staff member sends a short message — in, out, today's pieces, leave, an advance — and it becomes a real record: attendance with the time and place it was marked, a production report against the design, a request in the approvals queue. The end-of-day prompt asks what is still open and how long it needs, so tomorrow starts from what is actually pending rather than from memory. Marking attendance checks the phone is at the unit within the radius set for it, with a grace window; a person outside it is not refused, they are flagged for the manager, because a system that locks someone out of being paid for being early at the wrong gate has failed at its job. Every override is recorded with who made it. The words a worker types are in the language they speak, and nothing here ever asks for a password, a bank detail or a document number.
- **Data Privacy & Consent** — What a person's data may be used for, captured as consent at the point it is given and honoured everywhere downstream — including the right to have it corrected or removed. Retention (how long a record is kept) and deletion (a person's right to have their own data removed) are two different policies, tracked separately, because a rule that keeps records for the law and a request from a person to be forgotten do not resolve the same way.
- **Provider Router & Cost Guard** — The rule that no capability depends on one outside service, enforced at the moment it matters instead of merely promised. Every capability has an ordered fallback list ending on an option that needs nothing connected, so a courier API that stops answering at 9pm, or an AI key that hits its quota mid-catalogue, drops to the next option instead of stopping the work. A provider that keeps failing is tripped out of the list entirely and retried once after a cooldown rather than hammered; retries inside one provider wait twice as long each time. And every paid call is counted in paise against a ceiling you set — over

the ceiling the paid provider is refused, not warned about, and the work completes on a free one. Because every capability is guaranteed a built-in or by-hand option, a spent budget can stop the spending without ever stopping the business.

- **Payment Data Scope** — A written statement of exactly which systems ever see a card or bank credential, and which never do — because every card-capable screen in this system hands that moment to a payment provider's own secured field and never stores or even passes the number through application code. The statement is what an auditor or a partner asks for before they will connect to this system.

Module 02 · Design & Sampling — 2 apps

- **PLM & Development** — First idea to something you can actually make: specification, sample rounds, costed trials and sign-off, with every version kept. A product, a machined part, a formulation or a service package all move through stages you set yourself.
- **Design / IP Register** — What protects a design once it exists — trademark or copyright status, the date it was first shown, and a flag the moment a near-identical listing turns up elsewhere. Every version already kept by PLM & Development gets a filed status here instead of just a date stamp, because a design with no ownership record on file is a design nobody can defend.

Module 03 · Inventory & Catalog — 4 apps

- **Stock** — Live quantity by SKU, location and stage, with reorder alerts, batches, kits and dead-stock. Goods you still own but that sit in a channel's own warehouse are a location like any other, so consignment and sale-or-return stock is counted, valued and aged with everything else instead of disappearing off the books until it sells.
- **Catalog / PIM** — One product record — attributes, images, HSN, MRP and the price each channel actually sells at — pushed to every marketplace and to your own storefront, and scored for each channel's rules before it lists. It also holds the two things everything downstream depends on: the code each channel knows this product by, mapped to yours, and the packed size and weight that decide the courier rate and settle every weight dispute. Every listing's state on every channel is here too — live, waiting for your approval, blocked, archived — with the quality score that decides whether anyone sees it.
- **Kit & Combo SKU** — A sellable SKU made of component SKUs — a three-piece set sold as one listing. Selling the kit decrements each component at order time, so stock is right for every piece in the set, not just the set itself.
- **Master-Data Hygiene** — Duplicate detection and merge across customers, vendors and designs, and a dead-stock register for what has not moved in months. Protects every downstream report from the same record existing twice under two names.

Module 04 · CRM — 4 apps

- **CRM & Customer 360** — Lead to won, then the full lifetime: orders, returns, value and what to offer next.
- **Documents & eSign** — Every agreement, receipt, certificate and scan filed against the record it belongs to — an order, a party, a case, an employee — so it is found by that record instead of by remembering a folder. Send one out for signature and the signed copy files itself back.
- **Helpdesk & Live Chat** — Questions arriving by chat, email or phone become tickets tied to the order or the account they are about, with the whole history already on the screen.

- **Forms & Feedback (NPS)** — A short form after delivery, and the score it produces attached to the design or item it is actually about — not just the buyer — so a complaint-prone item surfaces as a pattern instead of a scatter of individual gripes.

Module 05 · Sales — 8 apps

- **D2C Sales** — Orders from your own storefront — Shopify, WooCommerce or a custom site — cart to dispatch, with loyalty and partial COD.
- **B2B & Credit** — Wholesale orders with credit limits, tier pricing and outstanding ageing.
- **Export** — Commercial invoice, packing list, LUT bond and IGST-refund tracking.
- **POS** — Counter billing that draws on the same stock as your website.
- **Quotes & Proforma** — Send a quote, convert it to a confirmed order in one click.
- **Couriers & AWB** — Book the shipment on the order itself, compare couriers, print the label and follow the AWB to the door.
- **Subscriptions** — A schedule that raises its own invoice on its cycle and follows up on its own when a payment fails — for anything sold as a standing order rather than a one-off.
- **Customisation & Made-to-Measure** — The order that does not exist in the catalogue: a buyer sends reference pictures and their own measurements, a price is agreed over several messages, and the piece is made for them. All of it is one record — the references, the measurement set, every quote in the negotiation and what was finally agreed, the advance taken to start work and the balance taken before dispatch. When the order is accepted it opens a production order like any other, so a bespoke piece is costed, made, checked and posted exactly as a catalogue piece is. Two legs of money on one order is the part most systems get wrong: the advance is earned when the work starts, the balance is owed until the piece ships, and the ledger shows both separately rather than one payment appearing when the whole thing is over.

Module 06 · Planning & Requirements (MRP) — 3 apps

- **Demand Forecast & Signal** — What sold, by SKU and by period, turned into a short-term forecast — the number every requisition and every production order downstream is measured against instead of a guess.
- **Requirement Explosion (MRP run)** — Confirmed demand exploded through the bill of materials into what raw material to buy and what to produce, and by when — so a purchase requisition or a production order always traces back to real demand, never to a feeling that stock is getting low.
- **Open-to-Buy / Budget Ceiling** — A spending ceiling set per period regardless of what the demand signal suggests, so a real signal cannot on its own commit more money than the business has decided to risk this season. Every requisition the MRP run drafts is checked against what is left of the ceiling before it goes to Purchase.

Module 07 · Purchase — 3 apps

- **Procurement** — RFQ to purchase order to goods receipt, with a strict three-way match before any bill is paid.
- **Vendor Management** — Vendor 360, payables, ageing, a real risk score, and sourcing that follows performance.
- **Insurance Register** — What cover exists over stock in transit, stock in the warehouse and product liability, matched against what is actually moving through Purchase and Warehouse right now — so real value in

transit is never quietly running with no cover tracked anywhere.

Module 08 · Manufacturing — 4 apps

- **Production Orders** — Your own stages from first operation to finished goods, with work-in-progress visible at each one.
- **Piece-rate & Contractors** — Output-based pay for anyone paid by the piece rather than the hour — pooled completion, per-unit rates, rework and advances resolved into a single payout.
- **BOM & Consumption** — What each product consumes, costed at today's material rates.
- **Maintenance** — Machines, tools and assets: what is due for service, when it was last done, what it cost, and what stopped while it was down. Planned and breakdown work against the same asset record.

Module 09 · Quality & Compliance — 2 apps

- **Quality Control** — Accept, reject or rework — on goods received and on goods made — with reasons that feed the supplier scorecard and the performance flags alike.
- **Certificate & Compliance Register** — Every standard the business or a vendor holds — a safety, labour or environmental certification — with its issue date, its expiry, and the audit that backs it, so a certificate about to lapse is visible before a buyer asks for it and finds it already expired. What this register tracks is what a sustainability report downstream is actually built from.

Module 10 · Warehouse — 3 apps

- **Picking & Bins** — Pick lists that tell staff exactly which bin to walk to, in walking order.
- **Barcode Operations** — Scan to pick, pack, dispatch and run a physical stock count from a phone — the same scan whether the order came from a marketplace, your Shopify site or the counter.
- **Packing Video** — Every parcel recorded as it is packed and indexed by its order number, so a wrong-item claim is answered with the clip. The footage attaches itself to the claim that needs it.

Module 11 · Logistics — 5 apps

- **Rates & Zones** — Every courier's rate card by zone, weight slab and service — so the cheapest and the fastest option for this parcel are both known before it is booked.
- **NDR & RTO Rescue** — A failed delivery worked while it can still be saved — reattempt, call, correct the address — before it becomes a return you pay for twice.
- **COD Remittance** — What the courier collected at the door against what reached your bank, parcel by parcel, with every shortfall named and aged.
- **Handover & Manifest** — What is expected out today against what the courier actually took, counted per courier and per service. The manifest to hand over, the one-time code to confirm it, and a signed record of the parcels that were left behind — so a parcel lost between your table and their van has an owner.
- **Fleet** — Your own delivery vehicles, if you run any — what each trip cost, what is due for service, and what that adds to freight. Optional; most businesses run couriers only.

Module 12 · Accounting & GST — 9 apps

- **Accounting** — Double-entry books where every voucher balances and the trial balance always ties.
- **Invoicing** — GST tax invoices and receipts, totals computed from the lines to the paise. Where a channel raises its own invoice, both numbers live on the order — theirs and yours — so the panel's paperwork and your books point at the same sale and neither has to be re-keyed to find the other.
- **Expenses** — Spend captured by category with approvals, and bill OCR to save typing.
- **GST & Tax** — CGST, SGST, IGST, TDS, TCS, input credit, GSTR-1 and GSTR-3B — filed per registration, so a group where one company is registered and another is not files exactly what each one owes and nothing it does not.
- **ITC Reconciliation** — Every input credit matched against the government's own GSTR-2A/2B before a return is filed, so a credit claimed is a credit that is actually on record.
- **Receivables, Payables & PDC** — Payments and receipts allocated against named open invoices, and a register for post-dated cheques that posts only on the date they are realised, not the date they were written.
- **Fixed Assets & Depreciation** — The asset register with both Straight-Line and Written-Down-Value depreciation tracked side by side, and disposal posting its gain or loss straight to the books.
- **Year-End Close & Period Lock** — Profit-and-loss accounts reset and balance-sheet accounts carry forward at year end, and a reviewed period locks against backdated edits until an admin unlocks it — an act that is itself logged.
- **Finance Reports** — P&L, balance sheet, and profit by channel, product and SKU.

Module 13 · Treasury & Financial Planning — 3 apps

- **Cash Flow Forecast** — Expected receipts and expected payments laid out by week, drawn from open invoices and open bills rather than typed in by hand — so a cash shortfall is visible weeks before the date it would actually bite.
- **Banking & Reconciliation** — Bank statement lines matched against the ledger's own record of what should have moved, with anything unmatched surfaced instead of silently carried forward.
- **Budget vs Actual** — A budget set per category and period, and actual spend from Accounting tracked against it as the period runs — not only after it closes, when the only thing left to do is explain the variance.

Module 14 · Settlement — 3 apps

- **Payout Cycles** — Every settlement cycle each panel runs — what it should pay, what actually landed in the bank, and on which day — so a late payout is visible the day it is late, not at month end.
- **Fee & Commission Audit** — The rate card a channel publishes against the rate it actually charged, category by category and SKU by SKU. A silent commission increase is caught the first time it is applied — and the tier you are rated in is on the same screen, because the tier is what the rate card hangs off, and losing one quietly costs more than any single deduction.
- **TCS & TDS Register** — Every rupee the panels deducted as TCS and TDS, matched against the portal's own figures — so the credit you claim is the credit you are actually owed.

Module 15 · E-commerce / OMS — 11 apps

- **Marketplace OMS** — Every marketplace and every storefront in one order queue — Amazon, Flipkart, Myntra, Meesho, Ajio, Nykaa and JioMart alongside Shopify, WooCommerce, Magento, Wix and your own custom site. The stages each channel really uses — to accept, to pack, ready to dispatch, handed over, in transit — with the right cut-off counting down on every order, because a quick-commerce or air-shipped order is not due at the same hour as a standard one. Priority orders first, the day grouped by product so one item is picked once instead of once per parcel.
- **Order Management** — One pipeline from new to delivered, whether the order came from a seller panel, your Shopify or WooCommerce site, a dealer or the counter.
- **Manual Data Check** — Upload the sheets you already download — marketplace orders and returns, and your own counter-shop registers, one file or a whole ZIP — and read ten cross-checks back: money, month, item, state, returns, claims, ads, payouts and GST. Every figure is clickable down to the transactions behind it, and the whole result downloads as Excel.
- **Reconciliation** — Match every marketplace payout to the order line that earned it, and expose the gap.
- **Claims & Disputes** — Turn shortfalls, weight disputes and lost parcels into filed claims with evidence — and answer them before the clock runs out. A claim that is awaiting your response is worth money; one closed for no response is worth nothing, so the days remaining sit on the screen next to the amount.
- **Returns / RMA** — Customer, courier and wrong returns — and the dead stock they actually cost you.
- **Channels & Storefronts** — Connect a channel once and it stays in step: catalogue out, price out, stock out, orders in. Seven marketplaces and any website platform — Shopify, WooCommerce, Magento, BigCommerce, Wix or a custom site over its own API — each switchable without touching your data. Shopsy and any other storefront a channel runs alongside its main one counts as its own channel here. Where a channel has no open interface, its own downloaded report is a first-class way in. A channel may also know you by a different trading name — that is a label on the channel, not a second company, so it tags the order and the payout without ever splitting your books.
- **Labels & Documents** — The channel gives you a PDF; this turns it into something a packer can work from. Cropped to your label size, your own product code printed large where the channel left it off, the invoice and the packing slip merged behind it, and the whole batch sent to the label printer in one job. Reprint a single parcel without redoing the batch — and nothing is ever uploaded to an outside website to be cropped.
- **Listing & Catalog Manager** — Bulk-create and bulk-edit listings across every channel from the one product record in Inventory & Catalog, and catch the mismatches that quietly cost sales: listed but out of stock, or in stock but never listed.
- **Size / Fit Recommendation AI** — A fit suggestion at the point of purchase, built from the item's own measurements and the return history of buyers who picked each size — aimed straight at the return reason that costs the most: the right item in the wrong size.
- **AR / Virtual Try-On** — A way to see drape, fit and colour on a screen before buying, for items where a flat photo alone leaves too much to guess.

Module 16 · HR & Payroll — 5 apps

- **Staff & Contractors** — Attendance, effective-dated salary and output-based earnings in a single register, whoever is on it.

- **Time-off & Advances** — Leave, festival advances, and exactly how they change this month's payout.
- **Appraisal & Hiring** — Performance reviews and a hiring pipeline that ends in an employee record.
- **Recruitment** — The pipeline before someone becomes an employee — an opening, the people who applied for it, a trial piece where the work itself is the interview, and the decision with its reason kept. It matters more in a skilled trade than in most: a person is taken on for skill at one particular kind of work, and the trial output is the evidence, so it is recorded against that work and the rate that would apply rather than remembered as an impression. A candidate who is not taken on now stays findable when the same skill is needed in a busy month, and their personal documents are held under the same consent and retention rules as anyone else's, not in a folder on somebody's phone.
- **Payout Execution** — Where the calculation in the earnings register actually turns into money leaving the business — bank batch, UPI, cash against a signed receipt — with the method and the reference recorded against every payout, so the register's total and the money that actually moved can always be checked against each other.

Module 17 · Marketing — 8 apps

- **Social Calendar** — Plan and publish across every channel from one calendar.
- **Campaigns** — Email, SMS and WhatsApp campaigns measured on real revenue, not opens.
- **Repricing Engine** — Rules per channel and SKU — floor, ceiling, match-lowest, festival overrides — and what each change actually did. A price that went up and took the orders down with it shows as exactly that, next to the rule that raised it, so the rule can be reversed on evidence rather than on a feeling.
- **Automation** — If this happens, do that — across any module, without writing code.
- **Blog & Pages** — Articles, landing pages and category copy written, scheduled and published straight to your own site — Shopify, WooCommerce, Magento or a custom CMS — with the meta title, description and internal links set before it goes out.
- **Events** — Trade shows and exhibitions worked as a channel of their own — booth, budget and every lead captured on the floor landing straight in CRM instead of on a stack of business cards.
- **Website & Page Builder** — The storefront itself, built by dragging sections into place rather than by editing a theme file — hero, product grid, size guide, lookbook, contact form — each block reading live from the catalogue, so a price or a stock state on a landing page is the same number the order screen uses instead of a figure someone pasted in and forgot. Blog & Pages above writes articles into a site that already exists; this is for the businesses that do not have one, and it is the gap that shows up plainly when this module list is set beside a mature open-source ERP: they ship a full site builder next to the blog, and until now this did not.
- **Markdown / Clearance Optimization** — The same rule engine that reprices for competitiveness, aimed at ageing stock instead: when to start discounting it and by how much, before it becomes a warehouse write-off rather than a sale at a lower margin.

Module 18 · AI Content Engine — 8 apps

- **Content Engine** — Fourteen stages in your own voice, from buyer research and competitor reading through hooks, channel-ready listings, ads, social posts, video scripts, song lyrics, the publishing calendar and alt text — each written from your own catalogue, so the words match the thing.

- **Image Studio** — Layers, free transform, background removal, channel presets and SEO alt text — a phone photo becomes a channel-compliant product image.
- **Video Studio** — Text and image to video, reels and ad cuts sized for every channel.
- **Design Studio** — A full design surface — templates, layers, undo and redo, any colour, exact sizing, background images and stock elements — exporting PNG, JPG or PDF at whatever size the channel or the printer asks for.
- **Motion Renderer** — A reel rendered from a page of HTML and CSS — the same layers, fonts and brand colours the Design Studio already uses — into a real MP4, on this machine, with nothing uploaded. It is not a screen recording: the clock is faked, the animation is seeked to the exact instant of each frame, and only then is that frame captured, so the render does not care whether the machine was busy. Rendering the same festival banner twice produces the same file to the byte, which is what makes a reel something you can check and re-cut rather than something you have to watch all the way through and hope about.
- **Narration Studio** — A voice over the reel, in the language the buyer actually speaks — the same script the Content Engine wrote, spoken. The default needs nothing installed and no key, because every modern browser can already speak; a self-hosted or cloud voice sits behind it as an interchangeable provider for anyone who wants a cloned or branded one. Long scripts are split at sentence boundaries and rejoined, so a two-minute description is not cut off at whatever limit a service imposes. A voice cloned from a real person is only ever used with that person's recorded consent, filed against them in Data Privacy & Consent like any other permission.
- **Image Generation Slot** — Generated imagery — a model on a background you do not have to shoot, a festival backdrop, a lifestyle scene — as a capability with interchangeable providers rather than a bet on one service: a queue of jobs, a preview while it works, inpainting to fix one region, upscaling and face correction. This one needs a graphics card. Image models cannot run on an ordinary office machine or on the container this system is built in, so what ships is the queue, the review screen and the provider slot, with the generating itself done by whichever engine you point it at — your own GPU box, or a cloud service, swapped without touching anything else. Said plainly here because the alternative is a screen that looks finished and produces nothing.
- **Publisher** — One push sends the finished listing, picture and copy everywhere it has to appear — your storefront, each marketplace, each social account — and reports back what actually went live and what was rejected, with the reason.

Module 19 · SEO, AEO & AIO — 3 apps

- **Technical SEO & Schema** — Structured data, sitemaps and page-level technical checks against your own storefront and content pages, so a search engine can read what a page is actually about instead of guessing from the text alone.
- **Answer-Engine Optimization** — Content shaped to be quoted directly by an answer box or a voice assistant — a clear, citable answer near the top of the page — rather than written only for a person to scroll through.
- **AI-Engine Visibility Tracking** — Whether and how this business is actually cited when someone asks an AI assistant a shopping question in this category, tracked over time — the same discipline as rank tracking, aimed at a newer kind of result page.

Module 20 · Projects & Collaboration — 7 apps

- **Projects & Cases** — A project, a case file, an engagement or a job — whatever your work is called. Stages you define, owners, deadlines, documents, billable time and real cost, all on one record the ledger can see.
- **Timesheets & Planning** — Who is on what this week, and the hours that actually went in — against a project, a case, a job or a machine. Billable and non-billable separated, so a rate card turns straight into an invoice and a real cost.
- **Approvals** — One queue for everything waiting on a yes — a purchase order, a discount, a leave day, a credit note, a payment. The rule that sent it there is on the screen next to it, and the decision goes to the audit record.
- **Forum** — Questions and answers that outlive a chat — for customers, dealers or staff — with the useful ones kept where the next person will actually find them.
- **Automation Studio** — The place a person builds “when this happens, do that” by dragging it out and watching it run — a trigger, the steps after it, a branch where the answer decides which way to go — over the same event stream every module already writes to. Marketing’s Automation aims that idea at campaigns; this is the general one, reaching any module: a payment marked short holds the next dispatch and opens a claim, a worker’s pooled output crossing a threshold raises the payout for approval, a return marked damaged writes off the piece and messages the buyer. Every run is kept — what fired it, each step, what each step returned — because an automation nobody can inspect afterwards is a rule the business cannot trust with its money.
- **Discuss** — Conversation attached to the record it is about: this order, this bill, this case. A year later the reason for a decision is still sitting next to the decision.
- **Knowledge Base** — A searchable internal wiki of standard operating procedures, scoped to the role it applies to, so how a task is meant to be done is written down once instead of carried in one person’s head.

Module 21 · Dashboard & BI — 5 apps

- **CEO Dashboard** — Cash, sales, stock, profit and alerts on one screen, refreshed as work happens.
- **Report Builder** — Drag the fields you want into a report and save it for the whole team.
- **Group Consolidation** — Several companies, one set of figures — sales, cash, stock and profit rolled up across every company you run, inter-company entries removed, with years of history to compare against. Add a company whenever the business grows one; nothing in the software caps the number, only the plan does. And a company with no tax registration of its own — a job-work arm, a new venture not yet registered — is a company like any other here, kept in the group figures without being dragged into a return it does not belong in.
- **Excel Dashboard Builder** — A full workbook — financial summary, HR, purchase, sales, inventory and production, GST, expenses — generated from the live records behind every other screen, with each company shown as its own row and a consolidated row that is a formula over them, never a separately typed total.
- **ESG / Sustainability Reporting** — Water usage, chemical compliance, waste and packaging, reported from the same certificate and audit records Quality & Compliance already keeps — so a sustainability report is a query over evidence already on file, not a separate exercise assembled once a year from scratch.

Module 22 · AI Assistant, Agents & Automation — 5 apps

- **AI Assistant** — Ask in your own words — “what did Myntra actually pay us last week, and what is still short?” — and get the answer with the rows it came from sitting underneath it, each one clicking through to the record. It reads the ledger, the stock table and the settlement lines the same way a report does, so the figure it gives is the figure the books give. When it cannot find the answer it says so and shows what it looked at; it never estimates a number and presents it as a fact, because a plausible wrong figure is far more expensive than an honest blank. It answers only from records the person asking is already allowed to open, so it can never become a way around permissions.
- **AI Chatbot** — The same engine turned to face the customer, on your own storefront and on WhatsApp: where is my order, will this size fit me, I want to return this. It reads the real order and the real size chart rather than a script written six months ago, and it will say “let me get someone” instead of guessing at anything about money, a refund or a complaint. The handover goes into the Module 04 Helpdesk queue with the whole conversation already attached, so the person picking it up starts where the customer left off instead of asking them to explain again. It never asks a customer for a card number, a bank detail or a password — that promise does not get a chatbot-shaped exception.
- **AI Agents** — A job rather than a question: “chase every unreconciled settlement line from last week and draft the claim for each.” The agent works out the steps, does them, and stops at the point where a person has to decide. It runs inside a scope you set — which records it may read, which it may write, and how much it may spend through the Module 01 Provider Router — and it cannot quietly widen that scope mid-run. Anything that moves money, files a claim, changes a price or sends a customer a message waits for a human yes.
- **Agent Guardrails & Run Log** — What each agent is allowed to touch, written down as a scope rather than trusted to a prompt, and every run recorded step by step: what started it, what it read, what it proposed, what a person approved, what it actually changed. Kept in the same audit trail as everything else, with the same absence of an off switch. An agent whose working nobody can inspect afterwards is not a colleague, it is an unexplained entry in the books.
- **Knowledge & Retrieval** — The index that makes the answers grounded: your own designs, rate cards, settlement files, standard procedures and past decisions, searchable so a reply quotes what is actually on file instead of what a model remembers about the trade in general. Permission-scoped at the row, so two people asking the same question get answers drawn only from what each of them may already see.

12.1 · Mark the ones you will actually use in your first month **WITH YOUR TEAM**

Nobody starts with all of it, and starting with all of it is the usual reason an ERP rollout stalls. Pick the handful that carry your daily work — the making side, people and pay, selling, and the books — and switch the rest off until you want them.

Changing it: Turn any module or app on or off from a date. Turning one off hides a menu; it never destroys a record.

Done when: The menu shows what this business uses this month, and the rest is waiting rather than gone.

Part 13 · Listings and content — the engine that writes them

Every design has to reach a customer as words and pictures: a listing on each marketplace, a post, a reel, an advertisement, a description that ranks. Doing that by hand is where a catalogue of several hundred designs quietly stops being listed at all.

The engine is analysis-first, and that is the whole design. It looks at the actual product before it writes a word — category, colour, fabric, craft, occasion — and everything after that is built on what it found rather than on a template with the name swapped. A description generated without looking is the same description every time, and a marketplace ranks it accordingly.

The pipeline, phase by phase, read from the engine's own specification at the moment this document was generated:

Phase	What it does
0	Buyer Psychology Engine
1	Market Intelligence
2	Viral Hook Engine
3	Content DNA
4	Product Content Engine
4C	Social Media (3-format Engine)
4D	Thumbnail Engine
5	Ad Variation Engine
6	Cinematic Video Engine
7	Suno Music Engine
8	Marketplace Listings
9	Scale Engine
10	Social Media Automation Planner
11	Complete Excel Output (9 Sheets)
12	Measurement Chart (Complete Data)
13	SKU Image Metadata Generator

16 phases, and the order is the point. Each one reads what the ones before it established, which is why the analysis phases come first and why skipping them does not save time — it produces output the later phases then build on top of nothing.

13.1 · Let it read the product before it writes about the product IN THE APP

The preflight is not a formality. It decides the category, checks the design has not already been written about in the same words, and picks the vocabulary — colour, fabric, craft, occasion — from what is actually there. Skipping it produces text that is fluent, generic and worthless, and the only way to tell the difference is to read it against the garment.

Careful. *If it cannot tell what the product is, it says so and stops. It does not write a plausible description of a garment it could not identify — that is the one output that costs you more than no output, because nobody checks the confident ones.*

Done when: Nothing is written before the product has been read, and a product it cannot identify produces a question rather than a paragraph.

13.2 · Ask for the channel you want, and get only that IN THE APP

A marketplace listing, a social post, a reel script and an advertisement are different things with different rules, different lengths and different fields. Asking for one and receiving all of them is not generosity — it is output nobody reads, and it buries the thing that was asked for.

You ask for	You get
A marketplace listing	That marketplace's own schema, its own field limits, nothing else
A social post	The post, in the formats that channel takes
A reel	The script, timed, with the music brief beside it
The full pack	Every channel, and only when you asked for every channel

Done when: A request for one channel produces one channel's output, in that channel's own shape.

13.3 · Refuse to publish anything with a placeholder in it IN THE APP

A listing that goes out with a bracketed placeholder where the fabric should be is worse than one that never went out. It is public, it is wrong, and it stays indexed. The check runs before delivery rather than after, which is the difference between a gate and a report.

The same check catches a measurement that was never given, a colour the engine could not name, and a claim about the product that nothing in the input supports. Each is reported by name.

Done when: Nothing reaches a channel carrying a placeholder, an unnamed colour, an absent measurement or a claim the product data does not support.

13.4 · Keep the brand words yours, and changeable WITH YOUR TEAM

The three labels do not speak the same way, and the difference is the point of having three. What each one sounds like is a setting — the tone, the vocabulary, the things it never says — and it belongs to you rather than to whoever configured the engine first.

Changing it: Tone, vocabulary and the never-say list are settings per label, effective-dated. Changing them today does not rewrite what was already published.

Done when: Each label reads as itself, the difference is written down as settings rather than held in somebody's head, and changing one changes nothing already live.

Generated by `brand/delivery/website/mktenant.js` from `brand/site/tenant.js`, `brand/site/tenantbuild.js` and this business's own recorded logic — the companies, the channel kinds, the set compositions, the column layout, the pay bases, the closed lists and the refusal checks are all read from source at generation time, never retyped. Nothing here is maintained by editing this file: edit the source and regenerate.

Vastrangam Group · Desire to Attire · Surat · Confidential